



NCIE-004

Technology Stack & Engineering Blueprint

In development — for human review — Version 1.1

Selects and governs technology families and engineering patterns implementing NCIE-002/003

Named vendors/products remain proposed defaults until institutionally approved

Traces to NCIE-001 (requirements), NCIE-002 v0.4 (architecture) and NCIE-003 v0.3 (canonical data)

Prepared: 02 September 2026

Maintained by: National Communications Intelligence Ecosystem (NCIE)

Document Governance

Document	NCIE-004 Technology Stack & Engineering Blueprint
Edition	In development — Version 1.0
Baseline authorized	NCIE-004 Skeletal v0.1 — approved for full expansion; 36-chapter structure locked
Requirements authority	NCIE-001 Master Product Requirements Document, Version 1.2
Architecture authority	NCIE-002 System Architecture & Technical Design, Version 0.4
Canonical data authority	NCIE-003 Data Model & Data Dictionary, Version 0.3 (Canonical Data Architecture Baseline)
Owner	National Communications Intelligence Ecosystem (NCIE) Architecture Function
Reviewer	David King Boison (PhD) — Academic Intelligence Center
Status	IN DEVELOPMENT — FOR HUMAN REVIEW until explicitly approved; named vendors/products are proposed defaults only

0.1 Authority Chain

NCIE-001 establishes what NCIE requires. NCIE-002 establishes how the system is architecturally structured. NCIE-003 establishes how the information that architecture requires is represented, related, governed and persisted. NCIE-004 selects and governs the technology families and engineering patterns that implement all three. NCIE-004 implements upstream authorities; it does not reinterpret them.

0.2 Token-Conservation Convention

This document does not reproduce NCIE-002/003 content. Every chapter opens with a short Upstream block citing the specific upstream chapter and stating only the engineering consequence for NCIE-004 — never a restatement of the upstream architecture itself. Where a reader needs the full upstream rationale, the citation is the pointer to go read it there.

0.3 Technology-Selection Status Classification

Every technology decision in this document is classified as one of five statuses. None of the last four is ever silently converted into a confident selection:

- APPROVED UPSTREAM MANDATE — already dictated by NCIE-001/002/003 or another approved institutional decision.
- PROPOSED DESIGN DEFAULT — a technically recommended implementation choice, subject to NCA confirmation.
- INSTITUTIONAL CONFIRMATION REQUIRED — depends on NCA enterprise standards, procurement, existing platforms or operational policy.
- INFRASTRUCTURE DISCOVERY REQUIRED — depends on actual NCA infrastructure, capacity, network, hosting or installed technology.
- SECURITY/SOVEREIGNTY CONFIRMATION REQUIRED — depends on classification, residency, legal, security or data-sovereignty requirements.

A named product (e.g. PostgreSQL, Kubernetes, Keycloak) is always a Proposed Design Default until institutionally approved — this document never asserts "NCIE uses X" as settled fact.

0.4 Governance Artifacts and Where They Live

Chapter 33 is the Technology Registry (governance of what has been approved: version, owner, license, support/security/lifecycle state). Chapter 35 is the Technology Decision Matrix (the choosing record: every Decision ID, capability, candidate/default, rationale, constraints, owner, status) plus the consolidated Human Review Register. Chapter 36 is the NCIE-002/003 traceability matrix and acceptance/handover gate. Individual chapters reference Decision IDs (e.g. TD-11-1) rather than repeating this content.

0.5 Status Convention

Skeletal-structure approval authorizes expansion; it does not approve any individual technology recommendation. Every chapter in this edition is stamped IN DEVELOPMENT — FOR HUMAN REVIEW and remains so until explicitly approved.

0.6 Hardening Pass (v1.1)

NCIE-004 v1.0 received a full human review. Disposition: CONDITIONAL PASS — architecturally sound, targeted technology-governance hardening required before baseline approval. Five corrections were requested and are applied in this v1.1 edition:

- (1) Every APPROVED UPSTREAM MANDATE technology-table row was audited against what NCIE-001/002/003 actually mandate; rows naming a specific implementation mechanism, algorithm or vendor-flavoured pattern beyond what upstream dictates were reclassified PROPOSED DESIGN DEFAULT (see Ch.33 §33.1A for the audit note and worked examples).
- (2) Chapter 35's Technology Decision Matrix and Human Review Register are extracted programmatically from the chapter tables at build time, not manually counted — the summary in Ch.35 §35.2 is mechanically correct by construction and cannot drift from the chapters it consolidates.
- (3) Chapter 35 §35.5 now states explicitly that Non-Blocking Proposed Design Defaults may support downstream design, prototyping, estimation and specification work, but do not constitute procurement, production or institutional technology approval.
- (4) Chapter 33 §33.3 adds the Registry promotion invariant: a Technology Registry entry never moves to Approved merely because its Proposed Design Default was used downstream; approval requires an explicit institutional decision against the Decision ID.
- (5) Chapter 33's seeded Technology Registry already stated version policies/support windows rather than pinned exact versions in v1.0; §33.2A now states this convention explicitly so it is not merely implicit.

Contents

This table of contents updates automatically in Microsoft Word. If page numbers do not appear immediately, select the field and choose Update Field.

- 0.1 Authority Chain 2
- 0.2 Token-Conservation Convention 2
- 0.3 Technology-Selection Status Classification 2
- 0.4 Governance Artifacts and Where They Live 2
- 0.5 Status Convention 3
- 0.6 Hardening Pass (v1.1) 3
- Engineering Mandate, Authority Chain & Technology Selection Doctrine 8
 - 1.1 Selection Doctrine..... 8
 - 1.2 Structural Change Control 8
 - 1.3 Boundary with NCIE-005 Through NCIE-016..... 8
 - Human Review Focus 8
- Engineering Principles, Standards & Reference Conventions 9
 - 2.1 Engineering Conventions 9
 - 2.2 Documentation and Traceability Convention 9
 - Human Review Focus 9
- Target Runtime, Hosting & Environment Strategy..... 10
 - 3.1 Hosting and Environment Model..... 10
 - 3.2 Failure and Recovery 10
 - Human Review Focus 10
- Containerization, Orchestration & Workload Scheduling 11
 - 4.1 Containerization and Orchestration..... 11
 - 4.2 Security and Failure..... 11
 - Human Review Focus 11
- Backend Application & Service Engineering Stack 12
 - 5.1 Backend Runtime and Service Framework 12
 - 5.2 Engineering Notes 12
 - Human Review Focus 12
- Frontend Web Application Stack 13
 - 6.1 Frontend Stack..... 13
 - 6.2 Engineering Notes 13
 - Human Review Focus 13
- Design System, Visualization & Geospatial Technology 14
 - 7.1 Visualization and Geospatial Stack..... 14
 - 7.2 Engineering Notes 14
 - Human Review Focus 14
- API Gateway & Service Communication Stack..... 15
 - 8.1 API and Service Communication Stack 15
 - 8.2 Engineering Notes 15

Human Review Focus 15

Event Streaming, Messaging & Asynchronous Processing 16

9.1 Event Streaming and Messaging Stack 16

Human Review Focus 16

Workflow, BPM & Long-Running Process Engine..... 17

10.1 Workflow Engine..... 17

10.2 Engineering Notes 17

Human Review Focus 17

Transactional Database Technology 18

11.1 Transactional Database 18

11.2 Engineering Notes 18

Human Review Focus 18

Analytical, Time-Series & Historical Data Platform 19

12.1 Analytical and Historical Platform 19

12.2 Engineering Notes 19

Human Review Focus 19

Object, Document & Evidence Storage 20

13.1 Object/Document Storage 20

13.2 Engineering Notes 20

Human Review Focus 20

Search, Indexing & Retrieval Stack 21

14.1 Search and Retrieval Stack..... 21

Human Review Focus 21

Graph Technology & Relationship Analytics 22

15.1 Graph Technology 22

Human Review Focus 22

Vector Retrieval & Semantic Search Technology 23

16.1 Vector Retrieval Stack..... 23

Human Review Focus 23

Cache & Distributed State Technology..... 24

17.1 Cache and Distributed State 24

17.2 Engineering Notes 24

ARGUS AI Platform & Model Gateway Technology 25

18.1 ARGUS Engineering Relationship 25

18.2 Model Gateway Technology 25

18.3 Classification-Aware Model Eligibility 26

Human Review Focus 26

AI Guardrails, Evaluation & Safety Engineering 27

19.1 Guardrails and Evaluation Stack..... 27

19.2 Engineering Notes 27

Human Review Focus 27

Agentic Orchestration & Tool Gateway Technology..... 28

 20.1 Governed Agentic Orchestration 28

 20.2 Tool Gateway 28

 20.3 Tool Gateway Enforcement Checklist..... 28

 Human Review Focus 29

Collaborative Intelligence & Real-Time Collaboration Stack..... 30

 21.1 Collaboration Stack — Not Chat Infrastructure 30

 21.2 Engineering Notes 30

Memory, Knowledge & Context Engineering Stack 31

 22.1 Memory Engineering Boundary 31

 Human Review Focus 31

Identity, Authentication & Access-Control Technology 32

 23.1 Identity and Access-Control Stack..... 32

 Human Review Focus 32

Secrets, Keys, Certificates & Cryptographic Services..... 33

 24.1 Secrets, Keys and Cryptographic Services..... 33

 24.2 Engineering Notes 33

 Human Review Focus 33

Data Protection, DLP, Masking & Protected Reveal..... 34

 25.1 Data Protection Stack..... 34

 25.2 Prohibited Exposure Surfaces 34

 Human Review Focus 34

Observability, Telemetry & Operational Monitoring Stack..... 36

 26.1 Observability Semantics 36

 Human Review Focus 36

Security Operations, SIEM & Threat Detection Integration 37

 27.1 Security Operations Stack..... 37

 Human Review Focus 37

DevSecOps, Source Control & CI/CD Engineering Toolchain 38

 28.1 DevSecOps Toolchain..... 38

 28.2 Engineering Notes 38

 Human Review Focus 38

Testing, Quality Engineering & Test Automation Stack 39

 29.1 Test Automation Stack..... 39

 Human Review Focus 39

Resilience, Backup, Disaster Recovery & Business Continuity Technology 40

 30.1 Resilience and DR Stack 40

 30.2 RTO/RPO by Tier 40

 Human Review Focus 40

Performance, Capacity, Scalability & Load Engineering 41

 31.1 Performance and Capacity Engineering 41

Human Review Focus 41

Integration Adapters, RPA & Legacy-System Engineering 42

 32.1 Integration and Legacy Engineering..... 42

 Human Review Focus 42

Engineering Governance, Technology Registry & Lifecycle Management 43

 33.1 Technology Registry vs Technology Decision Matrix 43

 33.1A Mandate/Implementation Separation (Audit Finding) 43

 33.2 NCIE Technology Registry (Seeded) 43

 33.2A Version Policy, Not Pinned Versions 44

 33.3 Lifecycle, Promotion and Deprecation 45

 Human Review Focus 45

Reference Deployment Topology & Environment Blueprint 46

 34.1 Technology Stack Layer Model 46

 34.2 Logical Deployment Topology..... 46

 34.3 Data Technology Topology 46

 Human Review Focus 47

Technology Decision Matrix, Proposed Defaults & Human Review Register 48

 35.1 Purpose 48

 35.2 Summary 48

 35.3 Technology Decision Matrix..... 48

 35.4 Human Review Register 56

 35.5 Prioritisation and the Limits of a Non-Blocking Default..... 60

NCIE-004 Acceptance, Traceability & Handover Gate 61

 36.1 Traceability Matrix 61

 36.2 Acceptance Checklist..... 62

 36.3 Technology Registry Snapshot..... 62

 36.4 Handover 62

 Human Review Focus 62

Engineering Mandate, Authority Chain & Technology Selection Doctrine

Governs technology selection under the approved NCIE authority chain.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.1 §1.3 — Human-Primary authority, provider neutrality, evidence/provenance-first bind every technology choice below. | NCIE-003 Ch.1-2 — Canonical semantics, temporal integrity and classification are engineering constraints, not optional features.

1.1 Selection Doctrine

- A technology is selected to implement an upstream requirement, never to justify one retroactively.
- Vendor/product selection is justified by security, sovereignty, resilience, maintainability and lifecycle — convenience alone is never sufficient.
- Every technology decision carries one of the five statuses in front matter §0.3; none is asserted as settled fact unless it is an Approved Upstream Mandate.
- This document distinguishes capability (what is needed) from candidate product (a proposed way to meet it) throughout — see Chapter 35 for the consolidated record.

1.2 Structural Change Control

The 36-chapter structure is locked. A material contradiction or omission discovered during expansion is flagged inline as STRUCTURAL CHANGE REQUIRED — HUMAN REVIEW rather than silently resolved by adding, merging or renumbering a chapter. No such flag has been raised in this edition.

1.3 Boundary with NCIE-005 Through NCIE-016

NCIE-004 establishes the technology baseline and engineering constraints for ARGUS (→ NCIE-005), memory (→ NCIE-006), AI/agent orchestration (→ NCIE-007), security architecture (→ NCIE-009), API/integration (→ NCIE-011), UX/UI (→ NCIE-012), database/storage engineering (→ NCIE-014), DevSecOps/infrastructure (→ NCIE-015) and testing (→ NCIE-016). It does not write those documents' implementation detail; it fixes the technology families and patterns they must build within.

Human Review Focus

- Confirm the acceptance authority for NCIE-004 as a whole and for individual chapter/technology sign-off.

Traceability: NCIE-002 Ch. 1; NCIE-003 Ch.1-2.

Engineering Principles, Standards & Reference Conventions

Sets enforceable engineering conventions across all implementation teams.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-003 Ch.2 — Naming, null/unknown semantics, temporal and classification rules bind every service's code and schema, not only the data model.

2.1 Engineering Conventions

Table 1. Engineering conventions applied across all NCIE-004 technology families.

Capability	Engineering Characteristics Required	Candidate/Default	Status
Code style & linting	Deterministic, automatically enforced, language-appropriate	Per-language formatter/linter (e.g. Black/Ruff for Python, ESLint/Prettier for TypeScript), enforced in CI (Ch.28)	PROPOSED DESIGN DEFAULT
API/error conventions	Consistent error shape, idempotency keys and correlation IDs on every call, per NCIE-003 Ch.33	Problem-details-style JSON error envelope; mandatory correlation ID header	PROPOSED DESIGN DEFAULT
Null/unknown representation	Must distinguish Unknown/Partial/Stale/Unavailable, never collapse to a default (NCIE-003 Ch.2 §2.5 mandates the vocabulary; the code-level mechanism does not)	Typed nullable/optional fields plus an explicit status enum	PROPOSED DESIGN DEFAULT
Versioning	Additive-first; breaking change requires major version + deprecation window	Semantic versioning across services and schemas	PROPOSED DESIGN DEFAULT

2.2 Documentation and Traceability Convention

Every service/component references the Decision ID (Ch.35) that selected its technology and the NCIE-002/003 chapter it implements, in its own README/architecture-decision-record — this keeps traceability alive in code, not only in this document.

Human Review Focus

- Confirm whether NCA already mandates a specific coding-standard/linting toolchain to adopt instead of §2.1's proposed defaults.

Traceability: NCIE-003 Ch.2, Ch.30 (dictionary/metadata conventions).

Target Runtime, Hosting & Environment Strategy

Defines sovereign hosting, environment tiers and runtime placement.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.1 §1.6 — Sovereign, on-premises-capable hosting is the proposed default posture; no core operation requires data to leave Ghanaian jurisdiction. | NCIE-003 Ch.32, Ch.36 — Hosting constraints gate storage-technology selection and sovereignty controls.

3.1 Hosting and Environment Model

Table 2. Hosting, environment and trust-zone decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Status
Hosting model	Sovereign, NCA-controlled, capable of on-premises or approved sovereign-cloud operation	NCA data centre or approved sovereign cloud within Ghana	SECURITY/SOVEREIGNTY CONFIRMATION REQUIRED
Environment tiers	Development / Test / Staging / Production, isolated, masked non-production data (NCIE-003 Ch.29 §29.3)	Four-tier environment model, one per NCIE-002 Ch.29	APPROVED UPSTREAM MANDATE
Trust zones	Network segmentation across ingestion/application/data/AI/integration boundaries is mandated (NCIE-002 Ch.26 §26.7); the specific zone taxonomy is an NCIE-004 engineering choice	The 7-zone model in §34.2 (edge/API, application, data, AI, integration, security-observability, management)	PROPOSED DESIGN DEFAULT
Capacity baseline	Sized to NCIE-002 Ch.30's proposed targets (500+ concurrent users, national cell/topology scale)	Sized once real infrastructure and volumes are confirmed	INFRASTRUCTURE DISCOVERY REQUIRED

3.2 Failure and Recovery

Loss of a single environment tier or trust zone must not corrupt another (NCIE-002 Ch.28). Recovery targets are inherited from NCIE-002 Ch.28 §28.9's proposed RTO/RPO tiers pending NCA confirmation (Ch.30 of this document details the resilience engineering).

Human Review Focus

- Confirm the specific hosting facility/sovereign-cloud provider and whether NCA already operates environment tiers this platform must integrate with.
- Confirm capacity/volume assumptions before infrastructure sizing (feeds Ch.31).

Traceability: NCIE-002 Ch.1 §1.6, Ch.26, Ch.28, Ch.30; NCIE-003 Ch.32, Ch.35-36.

Containerization, Orchestration & Workload Scheduling

Defines service packaging, orchestration, scaling and workload isolation.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.29 §29.3 — Performance/scalability targets require horizontal, autoscaling service deployment rather than fixed monoliths.

4.1 Containerization and Orchestration

Table 3. Containerization and orchestration technology decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
Container runtime	OCI-compliant, widely supported	containerd (via Docker build tooling)	CRI-O	PROPOSED DESIGN DEFAULT
Orchestrator	Declarative scheduling, autoscaling, rolling deploys, secrets integration (Ch.24)	Kubernetes	OpenShift (if NCA already standardises on it)	PROPOSED DESIGN DEFAULT
Ingress/networking	TLS termination, trust-zone-aware routing (Ch.3), rate limiting	NGINX/Envoy-based ingress controller	—	PROPOSED DESIGN DEFAULT
Autoscaling	Horizontal pod/service autoscaling on load and queue depth (Ch.9)	Kubernetes HPA + custom metrics	—	PROPOSED DESIGN DEFAULT
Stateful/GPU scheduling	Isolated scheduling for databases and AI model workloads (Ch.11-18)	Dedicated node pools with taints/tolerations; GPU device plugin where ARGUS requires local inference	—	INFRASTRUCTURE DISCOVERY REQUIRED

4.2 Security and Failure

- Workload identity per NCIE-002 Ch.4 §4.2 machine-identity model; no shared cluster-admin credentials for application workloads.
- Node/cluster failure: orchestrator reschedules; stateful workloads rely on Ch.30's HA/replication, not container restart alone.
- Monitored via Ch.26 observability stack (cluster and workload-level metrics).

Human Review Focus

- Confirm whether NCA already operates a container platform (Kubernetes distribution, OpenShift or other) that must be reused rather than newly stood up.
- Confirm GPU availability/capacity for any locally hosted ARGUS model (Ch.18).

Traceability: NCIE-002 Ch.4 §4.2, Ch.29, Ch.30.

Backend Application & Service Engineering Stack

Selects maintainable backend runtimes and service-engineering patterns.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.5-Ch.25 — Every domain and platform service (Network, Revenue, Rules, Evidence, etc.) needs one consistent backend runtime family.

5.1 Backend Runtime and Service Framework

Table 4. Backend application and service engineering decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
Primary backend runtime	Strong typing, mature async I/O, long-term support, maintainable by a growing NCA team	Python (type-checked) for domain/AI-adjacent services	TypeScript/Node.js (if frontend-adjacent team unification is preferred)	INSTITUTIONAL CONFIRMATION REQUIRED
API framework	OpenAPI-first, dependency injection, async request handling	FastAPI (Python)	NestJS (TypeScript)	PROPOSED DESIGN DEFAULT
Background workers	Reliable queue consumption, retries, idempotency (NCIE-003 Ch.23)	Framework-native worker process consuming the Ch.9 event broker	—	PROPOSED DESIGN DEFAULT
Dependency/build tooling	Reproducible builds, lockfiles, SBOM-capable (Ch.28)	Poetry/pip-tools (Python) or pnpm (Node.js)	—	PROPOSED DESIGN DEFAULT
Service template	Standard scaffold with logging, health checks, auth middleware pre-wired	Internal service template repository	—	PROPOSED DESIGN DEFAULT

5.2 Engineering Notes

- Every service exposes a health/readiness endpoint feeding Ch.26 observability.
- Service-to-service calls carry correlation IDs and enforce authorization per NCIE-002 Ch.4 at the boundary, not only at the API gateway (Ch.8).
- A service failing its readiness check is removed from the load-balancing pool by the orchestrator (Ch.4), never left serving degraded traffic silently.

Human Review Focus

- Confirm primary engineering language/runtime based on NCA's actual institutional engineering capability and long-term support commitments.

Traceability: NCIE-002 Ch.4, Ch.5, Ch.25.

Frontend Web Application Stack

Selects the web stack for dashboards, workspaces and collaborative interfaces.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.3 — The Situational Awareness Workspace, ARGUS presentation and Collaborative Rooms all render in one consistent web stack.

6.1 Frontend Stack

Table 5. Frontend web application stack decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
Web framework	Component-based, large ecosystem, SSR/CSR flexibility for dashboards and workspaces	React	Vue	PROPOSED DESIGN DEFAULT
Type system	Compile-time safety across a large, multi-team codebase	TypeScript	—	PROPOSED DESIGN DEFAULT
State/data layer	Server-state caching, optimistic updates for collaborative rooms (Ch.21)	React Query / equivalent data-fetching layer	Redux Toolkit (for complex local state)	PROPOSED DESIGN DEFAULT
Routing	Role-aware, deep-linkable views matching NCIE-002 Ch.3 panel model	Framework-native router	—	PROPOSED DESIGN DEFAULT
Build/accessibility tooling	WCAG 2.1 AA per NCIE-002 Ch.30 §30.4	Vite build; automated accessibility linting/testing in CI (Ch.28-29)	—	PROPOSED DESIGN DEFAULT

6.2 Engineering Notes

- Panel/workspace composition (NCIE-002 Ch.3) is implemented as independently loadable modules so a degraded panel never blocks the rest of the shell from rendering.
- Colour is never the only status signal, per NCIE-002 Ch.30 §30.4 — enforced via an accessibility lint rule, not convention alone.
- ARGUS presentation states (Docked/Expanded/Full-Screen/in-Room, NCIE-002 Ch.3 §3.9) are layout variants of one component, not separate applications.

Human Review Focus

- Confirm whether NCA has an existing design-system/frontend standard (e.g. from the NMS Dashboard) that should be reused rather than introducing a new one.

Traceability: NCIE-002 Ch.3, Ch.30 §30.4.

Design System, Visualization & Geospatial Technology

Standardizes UI components, charts, maps and spatial visualization.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.3, Ch.8 — The Situational Awareness Workspace's map-centrality and dashboard composition require a consistent visualization/GIS layer. | NCIE-003 Ch.8 — Coordinate reference system, spatial precision and sensitive-layer classification are canonical, not a rendering choice.

7.1 Visualization and Geospatial Stack

Table 6. Design system, visualization and geospatial technology decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
Design system/components	Consistent, accessible (WCAG 2.1 AA), shared across dashboards/workspaces/rooms	Internal component library on a headless UI kit (e.g. Radix)	Existing NCA design system, if any	PROPOSED DESIGN DEFAULT
Charting	Time-series, KPI and status visualizations matching NCIE-003 Ch.2 §2.5 quality states	Recharts/Observable Plot	—	PROPOSED DESIGN DEFAULT
GIS/map engine	Layered, permission-filtered rendering per NCIE-002 Ch.8 §8.6	MapLibre GL (open, avoids per-seat licensing)	Commercial GIS SDK if NCA already licenses one	INSTITUTIONAL CONFIRMATION REQUIRED
Spatial services	Coordinate transforms, geocoding matching NCIE-003 Ch.8's canonical CRS	PostGIS extension on the transactional store (Ch.11)	—	PROPOSED DESIGN DEFAULT
Dashboard/export rendering	Reproducible PDF/image export of workspace views for reporting	Server-side headless rendering	—	PROPOSED DESIGN DEFAULT

7.2 Engineering Notes

- Sensitive map layers (Anti-Fraud location, NCIE-003 Ch.15) are permission-filtered server-side before reaching the client, never hidden by client-side CSS alone.
- Map engine failure degrades to a non-map geography selector (NCIE-002 Ch.3 §3.6); the frontend must ship that fallback view, not merely assume map availability.

Human Review Focus

- Confirm whether NCA already licenses a commercial GIS/mapping SDK that should be reused instead of an open alternative.

Traceability: NCIE-002 Ch.3, Ch.8, Ch.30 §30.4; NCIE-003 Ch.8.

API Gateway & Service Communication Stack

Standardizes synchronous APIs, gateways and service-to-service communication.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.25 — Governed APIs, schema versioning and authorization-at-every-boundary are architectural mandates, not implementation options. | NCIE-003 Ch.33 — API/event contracts must carry knowledge time, lineage and classification, never strip them.

8.1 API and Service Communication Stack

Table 7. API gateway and service communication technology decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
Synchronous API protocol	OpenAPI-documented, broadly consumable by frontend/partners (NCIE-002 Ch.25 mandates governed, documented APIs; it does not mandate a specific protocol)	REST/HTTP+JSON	—	PROPOSED DESIGN DEFAULT
Internal high-throughput calls	Low-latency service-to-service where REST overhead matters (e.g. ARGUS tool calls)	gRPC where justified by measured latency need	—	PROPOSED DESIGN DEFAULT
API gateway	AuthN/AuthZ enforcement, rate limiting, request correlation, per NCIE-002 Ch.4/Ch.25	Kong / Envoy-based gateway	Cloud-native API Gateway if hosting model requires it	PROPOSED DESIGN DEFAULT
Schema tooling	Contract-first, breaking-change detection (NCIE-003 Ch.33 §33.4)	OpenAPI + schema-diff CI gate (Ch.28-29)	—	PROPOSED DESIGN DEFAULT
Idempotency enforcement	Idempotency-key on every side-effecting call, mandated verbatim (NCIE-003 Ch.33 §33.5)	Idempotency-key middleware in every service	—	APPROVED UPSTREAM MANDATE
Rate limiting	Prevents a single tenant/workload from starving others (NCIE-002 Ch.30 §30.3); mechanism/placement not itself mandated	Gateway-level rate limiting	—	PROPOSED DESIGN DEFAULT

8.2 Engineering Notes

- Every call carries a correlation ID; the gateway rejects a request without one at the edge rather than let it propagate uncorrelated.
- Authorization is enforced at each service boundary, not solely at the gateway, per NCIE-002 Ch.4's zero-trust principle.
- Gateway outage degrades to direct service health-check-based routing rather than a total outage where the orchestrator (Ch.4) permits.

Human Review Focus

- Confirm whether NCA's existing enterprise integration standards mandate a specific API gateway product.

Traceability: NCIE-002 Ch.4, Ch.25; NCIE-003 Ch.33.

Event Streaming, Messaging & Asynchronous Processing

Standardizes durable events, queues and asynchronous processing.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.25 §25.3-25.4 — Source/PM/Alert/Workflow/Collaborative events must be idempotent, correlated and versioned. | NCIE-003 Ch.28 — Event envelopes must carry correlation IDs and reference sensitive payloads rather than embed them.

9.1 Event Streaming and Messaging Stack

Table 8. Event streaming and asynchronous processing technology decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
Event broker	Durable, partitioned, replayable, supports the event categories in NCIE-002 Ch.25 §25.3	Apache Kafka (or Redpanda for lighter ops)	RabbitMQ (if throughput/replay needs are modest)	PROPOSED DESIGN DEFAULT
Queues	At-least-once delivery, dead-letter handling for workflow/automation events (NCIE-003 Ch.23)	Broker-native topics/queues	—	PROPOSED DESIGN DEFAULT
Schema registry	Versioned event schemas per NCIE-003 Ch.33	Confluent/Karapace-style schema registry	—	PROPOSED DESIGN DEFAULT
Dead-letter queue & replay	Failed events retained and replayable, never silently dropped, mandated verbatim (NCIE-002 Ch.25 §25.7); the DLQ/replay mechanism itself is an implementation choice	Broker-native DLQ + replay tooling	—	PROPOSED DESIGN DEFAULT
Outbox/inbox pattern	Exactly-once effect despite at-least-once delivery (idempotent consumption, NCIE-003 Ch.23 §23.6)	Transactional outbox on the publishing service's own database (Ch.11)	—	PROPOSED DESIGN DEFAULT

Figure 1. Event/workflow topology: publish-outbox-broker-consume-DLQ pattern feeding the workflow engine.

SOURCE	RELATIONSHIP	TARGET
Domain Service (publisher)	writes Outbox row + business change in one transaction, then	Outbox Relay
Outbox Relay	publishes to	Event Broker (partitioned, schema-versioned)
Event Broker	delivers to	Consumer Service (idempotent, correlation-ID checked)
Consumer Service	on failure routes to	Dead-Letter Queue (replayable)
Workflow Instance (Ch.10)	raises/consumes events via	Event Broker

Human Review Focus

- Confirm expected event throughput/retention before final broker sizing (feeds Ch.31).

Traceability: NCIE-002 Ch.25; NCIE-003 Ch.23, Ch.28.

Workflow, BPM & Long-Running Process Engine

Selects durable workflow technology for approvals and long-running processes.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.21 — Request/Run state machine, WAITING_FOR_PM-style persistent waits and human-task approvals must survive restarts. | NCIE-003 Ch.23 — Requested/Approved/Executed/Verified/Reconciling states are canonical entities the engine must expose, not internal engine state alone.

10.1 Workflow Engine

Table 9. Workflow/BPM engine technology decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
Workflow/orchestration engine	Durable execution surviving process/node restart, human-task support	Temporal	Camunda (BPMN-native, if NCA prefers visual process modelling)	PROPOSED DESIGN DEFAULT
State machine / BPMN	Explicit states matching NCIE-003 Ch.23 §23.2's four-state separation	Engine-native workflow definitions mapped 1:1 to canonical Workflow States	—	APPROVED UPSTREAM MANDATE
Timers & waiting dependencies	Persistent WAITING_FOR_PM/WAITING_FOR_TRAFFIC-style waits (NCIE-002 Ch.21 §21.4)	Engine-native durable timers/signals	—	PROPOSED DESIGN DEFAULT
Human tasks & approvals	Distinct approving actor from requester (separation of duties, NCIE-003 Ch.26)	Engine human-task API integrated with Ch.23 IAM roles	—	APPROVED UPSTREAM MANDATE
Reconciliation	Unknown outcome triggers reconciliation, never blind retry (NCIE-003 Ch.23 §23.2)	Engine-native compensation/reconciliation workflow	—	PROPOSED DESIGN DEFAULT

10.2 Engineering Notes

- Room-originated workflows (NCIE-002 Ch.20 §20.11) retain room/thread provenance as workflow metadata, not a side-channel record.
- Engine outage: in-flight workflows resume from persisted state on recovery (Ch.30); no workflow is lost to a process restart.

Human Review Focus

- Confirm which workflows are high-consequence and require mandatory dual approval before automated execution (populates Ch.35 Decision Matrix constraints).

Traceability: NCIE-002 Ch.21; NCIE-003 Ch.23.

Transactional Database Technology

Selects ACID storage for canonical transactional state.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.5 — Canonical/validated layers require ACID transactional storage; optimisation must never redefine canonical meaning. | NCIE-003 Ch.5, Ch.24, Ch.32 — Bitemporal storage, effective-dated Rules and the logical-to-physical map bind schema design here.

11.1 Transactional Database

Table 10. Transactional database technology decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
Relational DBMS	ACID, mature bitemporal/window-function support, row-level security	PostgreSQL	—	PROPOSED DESIGN DEFAULT
HA/replication	Synchronous replica for failover per Ch.30 RTO/RPO targets	Streaming replication + automated failover (e.g. Patroni)	Managed HA offering if hosting model (Ch.3) permits	PROPOSED DESIGN DEFAULT
Migrations	Versioned, reversible, reviewed schema changes (NCIE-003 Ch.1 §1.4 change control)	Alembic/Flyway-style migration tooling in CI (Ch.28)	—	PROPOSED DESIGN DEFAULT
Backup / PITR	Point-in-time recovery capability matching Ch.30 RPO is mandated; the specific WAL-based mechanism is PostgreSQL-oriented, not itself an upstream mandate	Continuous WAL archiving + scheduled base backups	—	PROPOSED DESIGN DEFAULT
Row/column security	Protected-identity field isolation is mandated (NCIE-003 Ch.14 §14.6, Ch.26); RLS + column encryption is one implementation of it, not the only one	Row-level security policies + column encryption (Ch.24-25)	—	PROPOSED DESIGN DEFAULT

11.2 Engineering Notes

- Bitemporal entities (NCIE-003 Ch.5 §5.5) use append-only versioned tables; no UPDATE ever destroys a prior version's row.
- Rule Versions (NCIE-003 Ch.24) live here as the authoritative store — never duplicated as literals elsewhere in the stack.
- Failure: automated failover to replica; application layer retries idempotently against the new primary (Ch.5 §5.2).

Human Review Focus

- Confirm whether NCA mandates a specific relational database platform already in institutional use.

Traceability: NCIE-002 Ch.5; NCIE-003 Ch.5, Ch.24, Ch.32.

Analytical, Time-Series & Historical Data Platform

Selects high-volume analytical and historical query platforms.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.30 — National-scale PM/Traffic/Revenue volumes require partitioned, query-efficient analytical storage distinct from transactional storage. | NCIE-003 Ch.32 §32.5, Ch.35 — Analytical stores are derived, never independently canonical; discrepancy from canonical source is a data-quality issue, not a competing truth.

12.1 Analytical and Historical Platform

Table 11. Analytical, time-series and historical data platform decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
OLAP/time-series engine	High-volume PM/Traffic observation storage and query (NCIE-003 Ch.7, Ch.11)	TimescaleDB (PostgreSQL extension, minimises polyglot footprint)	ClickHouse (if query volume later demands a dedicated columnar engine)	PROPOSED DESIGN DEFAULT
Warehouse/lakehouse	Cross-domain historical analytics and reporting	Deferred until analytical query volume justifies a separate warehouse	—	INFRASTRUCTURE DISCOVERY REQUIRED
Partitioning	Cross-period traceability across partitions is mandated (NCIE-003 Ch.35 §35.5); the specific partition key/scheme is an engineering choice	Native partitioning by period + operator	—	PROPOSED DESIGN DEFAULT
Materialized views	Derived views must be rebuildable and never independently canonical, mandated verbatim (Ch.32 §32.5); the refresh mechanism itself is an implementation choice	Scheduled materialized-view refresh	—	PROPOSED DESIGN DEFAULT
Historical query	Decision-time reconstruction support (NCIE-003 Ch.25)	Bitemporal query layer over Ch.11's canonical tables plus this analytical layer	—	PROPOSED DESIGN DEFAULT

12.2 Engineering Notes

- This layer is always rebuildable from Ch.11's canonical store; it is never the system of record for a fact.
- A materialized view found out of sync with canonical source triggers a Remediation Task (NCIE-003 Ch.29), not silent trust until next refresh.

Human Review Focus

- Confirm expected historical data volumes and query patterns before committing to a dedicated warehouse/lakehouse (currently deferred as Infrastructure Discovery Required).

Traceability: NCIE-002 Ch.30; NCIE-003 Ch.7, Ch.11, Ch.32, Ch.35.

Object, Document & Evidence Storage

Selects governed storage for documents, evidence artifacts and snapshots.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-003 Ch.17 — Evidence Objects require hash/integrity metadata, WORM-capable retention and chain-of-custody re-verification.

13.1 Object/Document Storage

Table 12. Object, document and evidence storage decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
Object/document storage	S3-compatible, versioned, encrypted at rest (Ch.24)	MinIO (self-hosted, sovereignty-friendly) or S3-compatible NCA-approved object store	Managed cloud object storage if hosting model permits	SECURITY/SOVEREIGNTY CONFIRMATION REQUIRED
Versioning	New-version-not-overwrite on correction is mandated (NCIE-003 Ch.17 §17.5); the storage mechanism is not	Object-store native versioning	—	PROPOSED DESIGN DEFAULT
WORM / immutability	Evidence cannot be silently altered post-registration (NCIE-003 Ch.17; capability named in the approved Ch.13 skeleton); the specific retention mechanism is a product feature, not itself mandated	Object-lock / compliance-mode retention on evidence buckets	—	PROPOSED DESIGN DEFAULT
Hashing	Chain-of-custody verification on registration and retrieval is mandated (NCIE-003 Ch.17 §17.6); the hash algorithm is not specified upstream	SHA-256 computed at registration and re-verified on formal retrieval	—	PROPOSED DESIGN DEFAULT
Malware scanning	Ingested artifacts (screenshots, uploaded files) scanned before acceptance	Scan-on-ingest pipeline ahead of validation (NCIE-003 Ch.6)	—	PROPOSED DESIGN DEFAULT

13.2 Engineering Notes

- A hash mismatch on retrieval halts release of the artifact and raises a chain-of-custody exception (NCIE-003 Ch.17 §17.7), never a silent pass-through.
- Lifecycle (hot/warm/cold tiering) follows NCIE-003 Ch.35 without changing meaning or classification.

Human Review Focus

- Confirm retention/WORM period requirements per NCA legal guidance (NCIE-003 Ch.17 §17-review already flags this upstream).

Traceability: NCIE-003 Ch.6, Ch.17, Ch.31, Ch.35.

Search, Indexing & Retrieval Stack

Selects authorization-aware full-text and faceted retrieval technology.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-003 Ch.36 §36.4 — Search indexes must apply classification-aware segmentation; embedding/indexing a protected field must never make it globally searchable.

14.1 Search and Retrieval Stack

Table 13. Search, indexing and retrieval technology decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
Search engine	Full-text + faceted search over canonical entities, authorization-aware	OpenSearch (avoids Elastic licensing constraints)	Elasticsearch if NCA already licenses it	PROPOSED DESIGN DEFAULT
Facets	Domain/classification/operator/date faceting matching NCIE-003 Ch.3 families	Engine-native aggregations	—	PROPOSED DESIGN DEFAULT
Authorization filters	Classification-aware filtering is mandated (NCIE-003 Ch.36 §36.4), which itself offers index segmentation, security metadata filtering or post-retrieval authorization as alternative mechanisms — this row selects one	Index-per-classification-tier + post-retrieval authorization check	—	PROPOSED DESIGN DEFAULT
Snippets	Result highlighting without leaking unauthorized field content	Engine-native highlighting restricted to authorized fields only	—	PROPOSED DESIGN DEFAULT
Reindexing	Rebuildable-from-canonical is mandated (Ch.32 §32.5); the specific pipeline mechanism is not	Scheduled/triggered reindex pipeline, never a manual one-off script	—	PROPOSED DESIGN DEFAULT

Human Review Focus

- Confirm autocomplete/search must never reveal restricted case names, SIM identifiers or locations (NCIE-003 Ch.4 §4.90) — verify engine config enforces this at index build time.

Traceability: NCIE-003 Ch.4 §4.89-4.93, Ch.36.

Graph Technology & Relationship Analytics

Selects technology for governed cross-domain graph relationships.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-003 Ch.19 — Typed edges require Edge Origin (observed/deterministic/human-inferred/ARGUS-proposed) and Validation State as first-class properties, not graph metadata bolted on.

15.1 Graph Technology

Table 14. Graph technology and relationship analytics decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
Graph engine	Property graph with temporal-edge support (NCIE-003 Ch.19 §19.3)	Neo4j	Apache AGE (PostgreSQL extension, minimises polyglot footprint)	PROPOSED DESIGN DEFAULT
Query language	Expressive multi-hop traversal with authorization-aware filtering	Cypher (Neo4j) / openCypher (AGE)	—	PROPOSED DESIGN DEFAULT
Temporal edges	Effective-period validity per NCIE-003 Ch.5, Ch.19 §19.3	Edge properties carrying effective-from/to, queried via Ch.5's bitemporal pattern	—	APPROVED UPSTREAM MANDATE
Access filtering	Sensitive traversal rules crossing protected-identity/Anti-Fraud boundaries (NCIE-003 Ch.19 §19.7)	Query-time authorization filter preventing traversal into unauthorized-classification nodes	—	SECURITY/SOVEREIGNTY CONFIRMATION REQUIRED
Graph analytics	Correlation pattern detection feeding Cross-Domain Fusion	Engine-native graph algorithms (community detection, shortest-path) as analyst-invoked queries, never autonomous	—	PROPOSED DESIGN DEFAULT

Human Review Focus

- Confirm sensitive graph-traversal rules with NCA IT Security/Anti-Fraud before enabling multi-hop queries in production (NCIE-003 Ch.19 §19-review).

Traceability: NCIE-003 Ch.5, Ch.19.

Vector Retrieval & Semantic Search Technology

Selects classification-aware semantic/vector retrieval infrastructure.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.26 §26.3 — Sensitive semantic/vector indexing requires index segmentation, security metadata filtering or post-retrieval authorization. | NCIE-003 Ch.36 §36.4 — Embedding a protected field must never make it globally searchable.

16.1 Vector Retrieval Stack

Table 15. Vector retrieval and semantic search technology decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
Embedding service	Classification-aware, routes through the Model Gateway (Ch.18), never a direct provider call	Embedding models registered in the Approved Model Registry (Ch.18)	—	APPROVED UPSTREAM MANDATE
Vector store	Metadata-filterable, index-isolated by classification tier	pgvector (PostgreSQL extension) for launch scale	Dedicated vector database (e.g. Qdrant/Milvus) if scale later demands it	PROPOSED DESIGN DEFAULT
Metadata filters	Classification/domain/room-scope filtering applied before similarity search returns results	Filtered ANN search restricted to the querying identity's authorized scope	—	APPROVED UPSTREAM MANDATE
Index isolation	Protected/sensitive content never in a general-purpose shared index (NCIE-003 Ch.36 §36.4)	Separate vector index per classification tier	—	SECURITY/SOVEREIGNTY CONFIRMATION REQUIRED
Re-embedding	Model upgrade must reconnect to governed state, not silently re-embed sensitive content without review	Scheduled, audited re-embedding pipeline	—	PROPOSED DESIGN DEFAULT
Evaluation	Retrieval quality assessed before production use (Ch.19)	Retrieval-quality evaluation harness integrated with Ch.19's AI evaluation suite	—	PROPOSED DESIGN DEFAULT

Human Review Focus

- Confirm which data classes may ever be embedded at all, versus excluded from vector retrieval entirely (feeds Ch.25 DLP policy).

Traceability: NCIE-002 Ch.26 §26.3; NCIE-003 Ch.36 §36.4.

Cache & Distributed State Technology

Defines cache and ephemeral distributed-state technology.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.30 §30.5 — Caching may never bypass authorization filtering or evidence/provenance recording; cache invalidation must preserve control correctness, not only data freshness.

17.1 Cache and Distributed State

Table 16. Cache and distributed-state technology decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
Distributed cache	Sub-100ms authorization-decision and rule-lookup caching (NCIE-002 Ch.4 §4.7, NCIE-003 Ch.24 §24.9)	Redis	—	PROPOSED DESIGN DEFAULT
Session/ephemeral state	Session tokens, presence (Ch.21), rate-limit counters	Redis-backed session store	—	PROPOSED DESIGN DEFAULT
Cache invalidation	No stale authorization decision past revocation is mandated verbatim (NCIE-002 Ch.4 §4.6); event-driven invalidation is one mechanism among alternatives (e.g. short TTL)	Event-driven invalidation on permission/rule change (Ch.9 event broker)	—	PROPOSED DESIGN DEFAULT

17.2 Engineering Notes

- Cache is never classification-isolated; a protected value is never cached without the same access control as its canonical source (NCIE-003 Ch.36).
- Cache-node failure degrades to direct database reads with higher latency, never to serving unauthenticated/unauthorized results.

Traceability: NCIE-002 Ch.4, Ch.30 §30.5.

ARGUS AI Platform & Model Gateway Technology

Implements ARGUS through a provider-neutral, policy-aware Model Gateway.

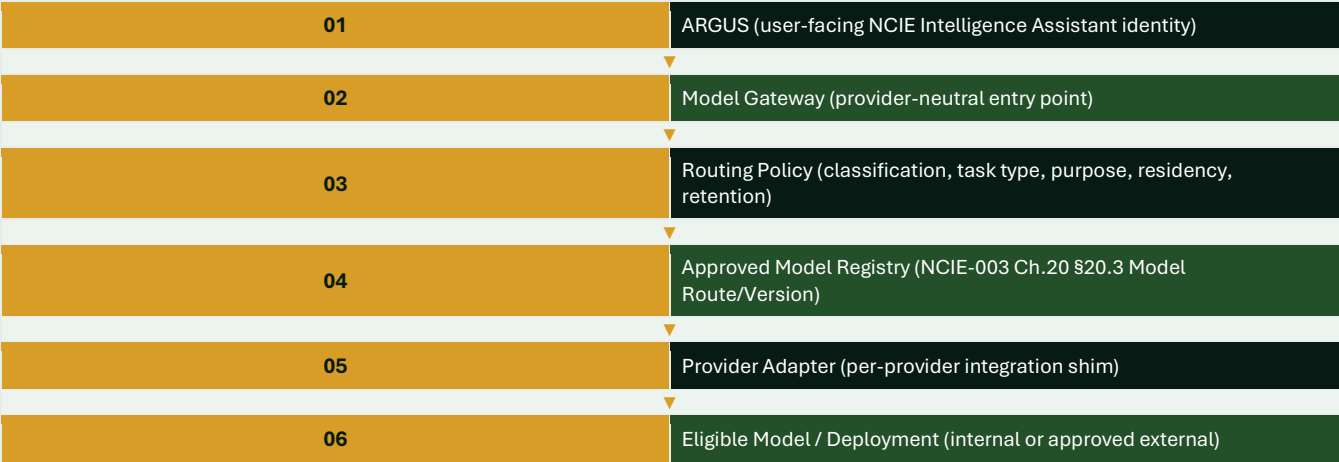
Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.19 §19.11, Ch.24 — ARGUS is a presentation identity for the NCIE Intelligence Assistant; the underlying architecture is provider-neutral via the Model Gateway. | NCIE-003 Ch.20 — ARGUS Interaction/Contribution/AI Provenance entities must be populated by whatever engineering implements the gateway, without persisting private chain-of-thought.

18.1 ARGUS Engineering Relationship

ARGUS is not a specific LLM. The engineering relationship is fixed and does not change with model/provider substitution:

Figure 2. ARGUS engineering relationship: ARGUS is never architected directly around one provider SDK.



18.2 Model Gateway Technology

Table 17. ARGUS Model Gateway technology decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
Model Gateway	Provider-neutral abstraction is mandated by name (NCIE-002 Ch.24 §24.2); this row states only that an internal gateway service must exist, not a specific implementation technology	Internal gateway service, provider-neutral by construction	—	APPROVED UPSTREAM MANDATE
Provider adapters	One adapter per eligible provider is the mandated pattern itself (NCIE-002 Ch.24 §24.2) — this is the mandate, not an implementation choice on top of it	Adapter-per-provider pattern behind a common interface	—	APPROVED UPSTREAM MANDATE
Approved Model Registry	That a registry tracking model/provider/classification eligibility exists is mandated by name (NCIE-003 Ch.20 §20.3); where/how it is stored is an implementation choice	Registry table in the transactional store (Ch.11), administered via Ch.33	—	PROPOSED DESIGN DEFAULT
Routing policy engine	Classification-aware eligibility per §18.3 below	Policy evaluated per-request before model selection	—	APPROVED UPSTREAM MANDATE

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
Context assembly	Minimal, authorization-filtered context per request (NCIE-002 Ch.24 §24.3)	Context Builder service filtering by requester's effective authorization	—	APPROVED UPSTREAM MANDATE
Fallback	Failover to an alternate approved model; never an unapproved model as fallback (NCIE-002 Ch.24 §24.9)	Routing policy defines an approved fallback chain per task type	—	PROPOSED DESIGN DEFAULT

18.3 Classification-Aware Model Eligibility

Routing considers data classification, task type, purpose, model capability, provider eligibility, residency, retention policy, tool requirements and availability together. An external model is never an automatic fallback for restricted information merely because the preferred (internal/sovereign) model is unavailable — unavailability degrades ARGUS conversational capability (Ch.19 §19.4 covers the outage boundary), it does not relax classification eligibility.

Human Review Focus

- Confirm internal-vs-external-vs-hybrid model deployment policy and data-residency constraints with NCA information-security and legal leadership (NCIE-002 Ch.24 §24-review already raised this).

Traceability: NCIE-002 Ch.19 §19.11, Ch.24; NCIE-003 Ch.20.

AI Guardrails, Evaluation & Safety Engineering

Engineers ARGUS guardrails, evaluation, regression and safe failure modes.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.24 §24.4-24.5 — Prompt-injection defense, secrets exclusion and collaborative-participation evaluation are mandatory before Participatory Mode production use.

19.1 Guardrails and Evaluation Stack

Table 18. AI guardrails and evaluation technology decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
Grounding controls	Answers derived from authoritative tool results, not unexamined model output (NCIE-002 Ch.19 §19.3)	Grounding Layer enforced in the Model Gateway response path (Ch.18)	—	APPROVED UPSTREAM MANDATE
Injection defenses	Ingested content (email, files) treated as data, never instructions (NCIE-002 Ch.24 §24.4)	Input-classification filter tagging untrusted content before it reaches a prompt	—	APPROVED UPSTREAM MANDATE
Evaluation harness	Collaborative-specific evaluation before Participatory Mode production use (NCIE-002 Ch.24 §24.5)	Offline eval suite (context tracking, attribution, thread separation, restraint) run per model/provider	—	APPROVED UPSTREAM MANDATE
Red-team testing	Adversarial testing for jailbreak, permission-pooling and privilege-escalation attempts	Scheduled red-team exercises ahead of each model/provider approval (Ch.33 registry gate)	—	PROPOSED DESIGN DEFAULT
Regression gates	A model/provider change cannot silently regress guardrail behaviour	CI gate blocking Model Registry promotion on failed evaluation suite (Ch.28-29)	—	PROPOSED DESIGN DEFAULT
Abstention	ARGUS states a limitation rather than fabricating an answer when ungrounded (NCIE-002 Ch.19 §19.8)	Grounding Layer returns an explicit "cannot verify" response class	—	APPROVED UPSTREAM MANDATE

19.2 Engineering Notes

- Over-participation and under-participation are monitored as distinct failure modes (NCIE-002 Ch.24 §24.5), each with its own alert in Ch.26 observability.
- No guardrail check may be bypassed for latency reasons; a guardrail-check failure blocks the response rather than degrading silently.

Human Review Focus

- Confirm acceptable red-team testing cadence and who reviews red-team findings before model/provider approval.

Traceability: NCIE-002 Ch.19, Ch.24.

Agentic Orchestration & Tool Gateway Technology

Implements governed agent/tool orchestration without expanding authority.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.21 — Room-to-workflow automation requests remain distinct from execution authority; AI suggestion never becomes execution. | NCIE-003 Ch.20 §20.5 — Every Tool Invocation must store its authorization-check result at invocation time, not just a success flag.

20.1 Governed Agentic Orchestration

Specialized agents may perform bounded analytical/tool tasks. Delegation to an agent never expands the delegating identity's authorization, data access, tool access or decision authority — an agent is not described or engineered as an autonomous institutional decision maker anywhere in this stack.

20.2 Tool Gateway

Table 19. Agentic orchestration and Tool Gateway technology decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
Tool Registry	Every tool explicitly registered with typed input/output before ARGUS/agents may call it	Internal Tool Registry service, versioned (Ch.33)	—	APPROVED UPSTREAM MANDATE
Tool Gateway enforcement	Authorization, classification, purpose and target validation on every invocation (NCIE-002 Ch.19 §19.7)	Gateway middleware enforcing the full checklist in §20.3	—	APPROVED UPSTREAM MANDATE
Agent orchestrator	Bounded task delegation without institutional authority expansion	Workflow-engine-hosted agent tasks (Ch.10), not a separate autonomous runtime	Dedicated agent-orchestration framework if task complexity later justifies it	PROPOSED DESIGN DEFAULT
Sandboxing	Agent-executed code/tools isolated from unrelated production workloads	Per-invocation isolated execution context (container/process, Ch.4)	—	PROPOSED DESIGN DEFAULT
Execution receipts	Idempotent, auditable record of what a tool actually did	Receipt written to Ch.27 (Audit) on every tool execution	—	APPROVED UPSTREAM MANDATE

Figure 3. Tool Gateway / Agentic Orchestration topology: every tool call is registered, checked, approved where required, sandboxed and receipted.

SOURCE	RELATIONSHIP	TARGET
ARGUS / Specialized Agent	requests tool via	Tool Gateway
Tool Gateway	validates against	Tool Registry (typed input/output, registered tools only)
Tool Gateway	checks	Authorization + Classification + Purpose + Target (§20.3 checklist)
Tool Gateway	requires Human Approval for	side-effecting tool calls (Ch.10)
Tool Gateway	executes within	Sandboxed context (Ch.4 isolation)
Tool Gateway	writes	Execution Receipt to Audit (Ch.27)

20.3 Tool Gateway Enforcement Checklist

- Tool registration (no ad hoc/unregistered tool calls).

- Typed input and typed output validation.
- Authorization check against the invoking identity's effective scope (NCIE-003 Ch.26).
- Classification check against the tool's declared data sensitivity.
- Purpose validation (is this tool call consistent with the current task/room context).
- Target validation (is the referenced object one the identity may act on).
- Human approval where the tool has a side effect requiring it (Ch.10).
- Idempotency key on every side-effecting call.
- Execution receipt and audit event on completion.

Human Review Focus

- Confirm which tool categories require mandatory human approval versus autonomous bounded execution.

Traceability: NCIE-002 Ch.19, Ch.21; NCIE-003 Ch.20, Ch.23.

Collaborative Intelligence & Real-Time Collaboration Stack

Implements persistent multi-user Rooms and human-ARGUS collaboration.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.20 — Human ↔ Human ↔ ARGUS collaboration requires persistent structured reasoning state, not merely a shared chat transcript. | NCIE-003 Ch.21 — Room, Thread, Hypothesis, Contradiction, Evidence Gap, Task, Human Judgment, Decision Point, ARGUS Intervention, Consensus, Dissent and Room Snapshot are canonical entities the stack must persist.

21.1 Collaboration Stack — Not Chat Infrastructure

Real-time messaging transport is only one component. The stack must persist the full structured-object set above as first-class, queryable records — a message log alone does not satisfy NCIE-003 Ch.21.

Table 20. Collaborative Intelligence technology decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
Realtime transport	Low-latency bidirectional updates for Room presence/events	WebSocket gateway (e.g. Socket.IO/native WS) behind the API gateway (Ch.8)	—	PROPOSED DESIGN DEFAULT
Presence	Participant join/leave/typing state per Room (NCIE-003 Ch.21 §21.3)	Presence service backed by cache (Ch.17)	—	PROPOSED DESIGN DEFAULT
Collaborative editing	Concurrent edits to shared reasoning objects without silent data loss (NCIE-002 Ch.20 §20.16)	CRDT-based or operational-transform sync for shared documents/boards	—	PROPOSED DESIGN DEFAULT
Whiteboard/evidence board	Shared references to authoritative Evidence, never copied authority (NCIE-002 Ch.20 §20.4)	Board component referencing Evidence Objects by ID only (Ch.13)	—	APPROVED UPSTREAM MANDATE
Room events	Material events (participation-mode change, promotion) individually audited (NCIE-003 Ch.27)	Room event bus on top of Ch.9's event broker	—	APPROVED UPSTREAM MANDATE
ARGUS hooks	Participation-mode-aware proactive contribution (NCIE-002 Ch.19 §19.4)	Room event subscription feeding the Proactive Contribution Controller (Ch.18/20)	—	APPROVED UPSTREAM MANDATE

21.2 Engineering Notes

- Every structured object (Hypothesis, Contradiction, etc.) is stored in the transactional store (Ch.11) with the same provenance/classification discipline as any other canonical entity — not a lighter-weight "collaboration database."
- Room Snapshot generation (NCIE-003 Ch.21 §21.9) reuses Ch.12's historical-query capability rather than a bespoke mechanism.

Traceability: NCIE-002 Ch.19, Ch.20; NCIE-003 Ch.20-21, Ch.27.

Memory, Knowledge & Context Engineering Stack

Implements persistent memory, institutional knowledge and context reconciliation.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.20 §20.2 — Private/Shared/Institutional memory tiers must never be an LLM conversation-history feature. | NCIE-003 Ch.22 — Memory ≠ Current Truth; Institutional Knowledge ≠ Rule; promotion requires recorded human review.

22.1 Memory Engineering Boundary

Memory is engineered as persistent, structured, versioned, authorization-aware, temporally aware, model-independent and reconcilable with current authoritative state — never as a model-provider conversation-history buffer that disappears on model/provider substitution.

Table 21. Memory and knowledge engineering technology decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
Memory service	Model-independence is mandated verbatim (NCIE-003 Ch.22 §22.3 / NCIE-002 Ch.20 §20.2); the storage location is not — this row proposes the transactional store over a provider-side feature	Service backed by the transactional store (Ch.11), not provider-side memory features	—	PROPOSED DESIGN DEFAULT
Knowledge repository	Institutional Knowledge's required attributes (Applicability Scope, Review State, Knowledge Version) are mandated (NCIE-003 Ch.22); its storage location is an implementation choice	Same transactional store, dedicated schema (Ch.11)	—	PROPOSED DESIGN DEFAULT
Dependency/context graph	Reconciliation of memory against current authoritative state (NCIE-003 Ch.22 §22.7)	Reconciliation job reusing Ch.15's graph technology where relational joins are insufficient	—	PROPOSED DESIGN DEFAULT
Snapshots	Room/session Snapshot capturing state for cross-session continuity is mandated (NCIE-003 Ch.21 §21.9); reusing Ch.12's query layer specifically is an implementation choice	Reuses Ch.12's historical/bitemporal query layer	—	PROPOSED DESIGN DEFAULT
Retrieval	Authorization-filtered retrieval, including any semantic/vector retrieval (Ch.16)	Retrieval layer enforcing Ch.16's classification-aware filtering	—	APPROVED UPSTREAM MANDATE

Human Review Focus

- Confirm memory retention periods by Memory Class with NCA (already an open item at NCIE-003 Ch.22 §22-review).

Traceability: NCIE-002 Ch.20 §20.2, §20.12; NCIE-003 Ch.22.

Identity, Authentication & Access-Control Technology

Implements IAM for humans, services, workloads and ARGUS.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.4 — Zero-trust, four-layer authorization (role/object/field/action) and distinct human/service/workflow/ARGUS identity classes are architectural mandates. | NCIE-003 Ch.26 — Effective-dated permissions and delegation must be queryable at any past point in time for decision-time reconstruction.

23.1 Identity and Access-Control Stack

Table 22. Identity, authentication and access-control technology decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
IdP/SSO	SAML/OIDC-capable, MFA-ready, integrates NCA's existing directory if one exists	Keycloak	NCA-existing IdP if one is already mandated	INSTITUTIONAL CONFIRMATION REQUIRED
MFA	Mandatory for Administrator, Investigator and regulatory validation actions (NCIE-002 Ch.4 §4.5)	TOTP authenticator app; SMS fallback disabled by default given SIM-fraud relevance	—	PROPOSED DESIGN DEFAULT
RBAC/ABAC/PBAC	Role plus contextual domain/object/field/action policy (NCIE-002 Ch.4 §4.3)	Policy engine (e.g. OPA) fronting the four-layer authorization model	—	PROPOSED DESIGN DEFAULT
Workload identity	Distinct machine identity for services/workflows/ARGUS (NCIE-002 Ch.4 §4.2)	SPIFFE/SPIRE-style workload identity or IdP-issued service accounts	—	PROPOSED DESIGN DEFAULT
Privileged access management (PAM)	Break-glass access logged, time-bounded and justified is mandated verbatim (NCIE-002 Ch.26 §26.4); a dedicated PAM module is one way to implement it, not the only one	PAM module on top of the IdP with mandatory justification capture	—	PROPOSED DESIGN DEFAULT
Federation	Single sign-on across all NCIE services and any existing NCA systems	IdP-federated SSO across the service mesh (Ch.8)	—	PROPOSED DESIGN DEFAULT

Figure 4. Security/trust-zone topology: identity classes and their authorization path across trust zones.

SOURCE	RELATIONSHIP	TARGET
Human User	authenticates via	IdP/SSO (MFA-enforced)
Service/Workflow/ARGUS	authenticates via	Workload Identity (distinct credential lifecycle)
Authorization Service (Ch.8 gateway + service boundary)	evaluates	Role → Object → Field → Action policy
Trust Zones (Ch.3)	bound by	Network segmentation matching identity class

Human Review Focus

- Confirm NCA's existing enterprise identity provider and role model before selecting/standing up a new IdP.
- Confirm step-up/MFA and break-glass policy requirements per role.

Traceability: NCIE-002 Ch.4, Ch.26; NCIE-003 Ch.26.

Secrets, Keys, Certificates & Cryptographic Services

Implements secrets, keys, certificates and cryptographic lifecycle controls.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.26 §26.7 — Encryption at rest/in transit is mandatory for protected/sensitive tiers; keys are managed centrally, never embedded in application code. | NCIE-003 Ch.36 — Encryption and masking are distinct controls; key management failure must fail closed, never fall back to an unencrypted path.

24.1 Secrets, Keys and Cryptographic Services

Table 23. Secrets, keys, certificates and cryptographic services decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
Secrets vault	Runtime secret injection by reference; never in source control or CI logs (NCIE-002 Ch.29 §29.6)	HashiCorp Vault	Cloud-native secrets manager if hosting model requires it	PROPOSED DESIGN DEFAULT
KMS/HSM	Centralised key management for protected/sensitive-tier encryption (NCIE-002 Ch.26 §26.7)	Vault Transit engine or dedicated HSM where sovereignty/compliance requires hardware-backed keys	—	SECURITY/SOVEREIGNTY CONFIRMATION REQUIRED
PKI / certificate automation	Automated issuance/rotation for service-to-service TLS	cert-manager (Kubernetes-native) with an internal or NCA-approved CA	—	PROPOSED DESIGN DEFAULT
Rotation	Scheduled, auditable, non-manual-only rotation is mandated as a capability; automating it via the specific vault/cert-manager tooling named in §24.1 is this document's proposed mechanism, not an upstream mandate	Automated rotation policies in the vault/cert-manager	—	PROPOSED DESIGN DEFAULT
Encryption/signing	Field-level encryption for protected identifiers is mandated verbatim (NCIE-003 Ch.14 §14.6); envelope encryption via KMS-issued keys is one implementation of it	Application-layer envelope encryption using KMS-issued data keys	—	PROPOSED DESIGN DEFAULT

24.2 Engineering Notes

- A KMS/HSM outage fails closed for protected-tier decryption operations — no application falls back to an unencrypted or locally-cached-key path (NCIE-003 Ch.36 §36.6).
- Secrets are referenced by name/path in configuration, never by literal value, across every chapter's technology stack.

Human Review Focus

- Confirm encryption/key-management standards and whether hardware-backed HSM is required by NCA policy versus a software-based KMS.

Traceability: NCIE-002 Ch.26, Ch.29 §29.6; NCIE-003 Ch.14 §14.6, Ch.36.

Data Protection, DLP, Masking & Protected Reveal

Implements masking, DLP, tokenization and governed protected reveal.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-003 Ch.36 §36.3-36.4 — Ghana Card/Passport identifiers and precise Anti-Fraud location require masking/tokenization, never plaintext exposure in logs, search, vector indexes or uncontrolled exports.

25.1 Data Protection Stack

Table 24. Data protection, DLP and masking technology decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
Tokenization	Reversible-only-under-authorization substitution for Ghana Card/Passport values is mandated (NCIE-003 Ch.14 §14.6); which tokenization service implements it is not	Vault Transit tokenization (Ch.24) or a dedicated tokenization service	—	PROPOSED DESIGN DEFAULT
Masking	Default-masked display for protected fields outside authorized Protected Reveal, enforced non-bypassably (server-side, never client-side only)	Server-side masking applied before response serialization, never client-side only	—	APPROVED UPSTREAM MANDATE
Field encryption	Encryption for protected columns is mandated (Ch.11, Ch.24); the specific envelope-encryption/KMS-key mechanism is an implementation choice	Application-layer field encryption using KMS-issued keys	—	PROPOSED DESIGN DEFAULT
DLP	Blocks bulk export of protected identity information (NCIE-003 Ch.36 §36.3)	DLP policy on export/API-response paths flagging bulk protected-field extraction	—	SECURITY/SOVEREIGNTY CONFIRMATION REQUIRED
Protected reveal	Every unmask is an audited action (NCIE-003 Ch.36 §36.5)	Protected-reveal endpoint requiring explicit authorization check + audit event (Ch.27)	—	APPROVED UPSTREAM MANDATE
Export controls	Anti-Fraud location/evidence exports individually authorized and logged	Export gateway enforcing classification + Ch.27 audit on every export	—	APPROVED UPSTREAM MANDATE

25.2 Prohibited Exposure Surfaces

- Raw Ghana Card/Passport values never appear in application logs, general search indexes, general vector indexes, ARGUS prompts (beyond the minimum authorized reference), telemetry, or uncontrolled exports — enforced by the masking/tokenization layer sitting ahead of every one of those surfaces, not by developer discipline alone.
- Anti-Fraud location data is never treated as ordinary GIS data (Ch.7); it inherits sensitive-tier isolation, authorization and ARGUS context-minimization (Ch.18 §18.3) automatically via its classification tag.

Human Review Focus

- Confirm field-level protection scope (which roles may invoke Protected Reveal) with NCA Legal/Data Protection (already open at NCIE-003 Ch.14 §14-review, Ch.31-review).

Traceability: NCIE-002 Ch.26; NCIE-003 Ch.14, Ch.31, Ch.36.

Observability, Telemetry & Operational Monitoring Stack

Standardizes logs, metrics, traces, SLOs and operational observability.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.27 — Platform, automation, source and sector health must remain distinct, individually visible signals. | NCIE-003 Ch.37 — Pipeline health is not sector health; source update time and processing time must remain distinct.

26.1 Observability Semantics

Five signal categories are kept separate throughout this stack and never collapsed into one dashboard colour: Platform Health, Data Pipeline Health, Sector/Network Condition, ARGUS Quality, and Security Condition. A green infrastructure dashboard must never imply that Ghana's telecommunications environment is itself healthy.

Table 25. Observability and telemetry technology decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
Logs	Structured, correlation-ID-tagged, classification-aware logging (no protected values in plaintext) is mandated; JSON format and log-store choice are not	Structured JSON logging shipped to a central log store	—	PROPOSED DESIGN DEFAULT
Metrics	Per-service and per-signal-category metrics (§26.1's five categories)	Prometheus-compatible metrics	—	PROPOSED DESIGN DEFAULT
Traces	Correlated distributed tracing across service/event boundaries (Ch.8-9)	OpenTelemetry instrumentation	—	PROPOSED DESIGN DEFAULT
APM/dashboards	Distinct dashboards per signal category (§26.1), never one merged view	Grafana dashboards, one board family per category	—	PROPOSED DESIGN DEFAULT
Alerting	Routes to the correct owner per signal category (platform vs sector vs security)	Alertmanager routing rules keyed to signal category and severity	—	PROPOSED DESIGN DEFAULT
SLOs	Per NCIE-002 Ch.30's proposed performance targets	SLO burn-rate alerting on the reference targets	—	PROPOSED DESIGN DEFAULT

Figure 5. Observability topology: five signal categories routed to distinct dashboards and owners.

SOURCE	RELATIONSHIP	TARGET
Services/Workloads (Ch.4-24)	emit logs/metrics/traces to	OpenTelemetry Collector
Collector	routes to	Metrics Store / Log Store / Trace Store
Metrics/Log/Trace Stores	feed	Dashboards (one family per signal category)
Dashboards	trigger	Alerting (routed by signal category to the correct owner)

Human Review Focus

- Confirm operational owners and escalation paths per signal category (already open at NCIE-003 Ch.37 §37-review).

Traceability: NCIE-002 Ch.27; NCIE-003 Ch.29, Ch.37.

Security Operations, SIEM & Threat Detection Integration

Integrates NCIE security telemetry with SOC/SIEM/threat-detection controls.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-003 Ch.27 — Every read/reveal/export/approve/execute action on sensitive data is an Audit Event; unknown outcome is a valid, first-class audit state.

27.1 Security Operations Stack

Table 26. Security operations and SIEM integration decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
SIEM	Ingests audit events (Ch.11), guardrail events (Ch.19) and infrastructure logs (Ch.26)	Wazuh (open) or NCA-existing SIEM	Commercial SIEM if NCA already operates one	INSTITUTIONAL CONFIRMATION REQUIRED
SOAR	Automated response playbooks for common security events	SIEM-integrated SOAR module where the chosen SIEM supports it	—	PROPOSED DESIGN DEFAULT
WAF/IDS/IPS	Perimeter protection at trust-zone boundaries (Ch.3, Ch.23)	WAF at the API gateway (Ch.8); network IDS/IPS at zone boundaries	—	PROPOSED DESIGN DEFAULT
EDR integration	Endpoint detection on hosts/nodes underlying the orchestrator (Ch.4)	EDR agent feeding SIEM	—	INFRASTRUCTURE DISCOVERY REQUIRED
Security-event pipeline	Fail-closed on high-consequence action audit failure is mandated verbatim (NCIE-003 Ch.27 §27.5); routing security events via Ch.9's broker specifically is this document's proposed mechanism	Same event broker (Ch.9) carrying a dedicated security-event topic into the SIEM	—	PROPOSED DESIGN DEFAULT

Human Review Focus

- Confirm whether NCA operates an existing SOC/SIEM this platform must integrate with rather than stand up independently.

Traceability: NCIE-002 Ch.26; NCIE-003 Ch.27.

DevSecOps, Source Control & CI/CD Engineering Toolchain

Defines secure source control, CI/CD, artifact and software-supply-chain tooling.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.29 — Environment separation, versioned deployments, and secrets excluded from code/CI logs are architectural mandates.

28.1 DevSecOps Toolchain

Table 27. DevSecOps and CI/CD engineering toolchain decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
Source control	Branch/review policy enforced, no direct pushes to protected branches	Git (GitLab/GitHub, self-hosted or NCA-approved)	—	INSTITUTIONAL CONFIRMATION REQUIRED
CI/CD	Automated build/test/deploy with mandatory gate checks (NCIE-002 Ch.29 §29.4)	GitLab CI or GitHub Actions, self-hosted runners in the sovereign environment (Ch.3)	—	PROPOSED DESIGN DEFAULT
Artifact/container registry	Immutable, versioned build artifacts (NCIE-002 Ch.29 §29.2)	Self-hosted registry (e.g. Harbor) inside the sovereign environment	—	PROPOSED DESIGN DEFAULT
SAST/SCA/secret scanning	Blocking a release with a critical vulnerability or leaked secret is the mandated gate; the named scanning tools are candidate products, not upstream mandates	Integrated scanners in the CI pipeline (e.g. Semgrep, Trivy, gitleaks)	—	PROPOSED DESIGN DEFAULT
IaC	Reproducible, reviewed infrastructure definitions	Terraform / Kubernetes manifests under source control	—	PROPOSED DESIGN DEFAULT
Policy-as-code	Enforces this document's mandates (e.g. no unencrypted protected-tier store) at deploy time	OPA/Conftest policy gates in CI	—	PROPOSED DESIGN DEFAULT
SBOM	Software bill of materials for supply-chain accountability where appropriate	Generated per release build (e.g. Syft)	—	PROPOSED DESIGN DEFAULT

28.2 Engineering Notes

- A failed mandatory gate (SAST/SCA/secret-scan/policy) blocks promotion; no manual override without a recorded exception (NCIE-002 Ch.29 §29.7).
- Rollback to the immediately prior artifact version is a tested, supported operation, not emergency improvisation (NCIE-002 Ch.29 §29.4).

Human Review Focus

- Confirm whether NCA mandates a specific source-control/CI platform already in institutional use.

Traceability: NCIE-002 Ch.29.

Testing, Quality Engineering & Test Automation Stack

Defines automated functional, data, security, performance and AI testing.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.31 — Test layers span functional, integration, security, AI/collaborative evaluation, automation, resilience and cross-domain/multi-user scenarios. | NCIE-003 Ch.38 — Data testing requires dedicated temporal, lineage and authorization/privacy test categories, not schema tests alone.

29.1 Test Automation Stack

This chapter defines the engineering toolchain and technical test capability only; the comprehensive verification programme (acceptance authorities, sign-off matrices, full RTM) is NCIE-016's scope, not reproduced here.

Table 28. Testing and quality-engineering technology decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
Unit/integration	Fast, deterministic, run on every commit	Language-native test frameworks (pytest/Jest) in CI (Ch.28)	—	PROPOSED DESIGN DEFAULT
Contract testing	API/event schema compatibility (Ch.8-9) verified pre-merge	Consumer-driven contract tests (e.g. Pact) or schema-diff gate	—	PROPOSED DESIGN DEFAULT
End-to-end/UI automation	Critical workspace/collaborative-room flows (Ch.6, Ch.21)	Playwright	—	PROPOSED DESIGN DEFAULT
Performance testing	Load/capacity validation against Ch.31 targets	k6 or similar load-testing tool in a staging environment	—	PROPOSED DESIGN DEFAULT
Security testing	SAST/SCA (Ch.28) plus scheduled DAST/pen-testing	DAST tool integrated pre-release; periodic external pen-test	—	PROPOSED DESIGN DEFAULT
Data testing	Temporal/lineage/authorization test categories (NCIE-003 Ch.38 §38.4)	Test harness asserting bitemporal correctness and field-level authorization per entity family	—	APPROVED UPSTREAM MANDATE
Workflow/resilience testing	Durable-workflow recovery (Ch.10) and failover drills (Ch.30)	Chaos/failure-injection testing in staging	—	PROPOSED DESIGN DEFAULT
AI evaluation	Model/provider evaluation before production use (Ch.18-19)	Offline evaluation harness gating Model Registry promotion (Ch.33)	—	APPROVED UPSTREAM MANDATE

Human Review Focus

- Confirm required test environments and masked test data provisioning (already open at NCIE-003 Ch.38 §38-review).

Traceability: NCIE-002 Ch.31; NCIE-003 Ch.38.

Resilience, Backup, Disaster Recovery & Business Continuity Technology

Defines HA, backup, DR, recovery testing and business-continuity technology.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.28 — Loss of any single external dependency (AI, RPA, map, source) must degrade only the affected capability, never core NCIE. | NCIE-003 Ch.5, Ch.25 — Backup/DR must preserve bitemporal history and provenance, not merely current-state snapshots.

30.1 Resilience and DR Stack

Table 29. Resilience, backup and disaster-recovery technology decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
HA/replication	Per-tier HA matching Ch.11's database replication and Ch.4's orchestrator rescheduling	Multi-replica deployment per service tier	—	PROPOSED DESIGN DEFAULT
Backup	Canonical data, rules, audit, memory, schedules and workflows all covered (NCIE-002 Ch.28 §28.4)	Automated scheduled backups per store (Ch.11-17)	—	APPROVED UPSTREAM MANDATE
PITR	Point-in-time recovery capability matching RPO targets is mandated; WAL archiving is a PostgreSQL-specific mechanism, not itself an upstream mandate	Continuous WAL archiving (Ch.11) + object-store versioning (Ch.13)	—	PROPOSED DESIGN DEFAULT
Cross-site DR	Secondary site/capability for platform-level failure	Warm-standby DR site within Ghanaian jurisdiction (Ch.3 sovereignty posture)	—	INFRASTRUCTURE DISCOVERY REQUIRED
Failover	Automated or documented manual failover per critical service tier	Orchestrator-driven failover (Ch.4) + database failover (Ch.11)	—	PROPOSED DESIGN DEFAULT
Restore testing	Recovery is rehearsed, not assumed (NCIE-002 Ch.29 §29.9)	Scheduled DR/restore drills with recorded results	—	APPROVED UPSTREAM MANDATE

30.2 RTO/RPO by Tier

Inherits NCIE-002 Ch.28 §28.9's proposed defaults: Tier 1 (identity, canonical data, evidence/audit) RTO 4h/RPO 15min; Tier 2 (domain services, automation) RTO 8h/RPO 1h; Tier 3 (AI/collaborative features) RTO 24h/RPO 4h — pending NCA business-continuity confirmation.

Human Review Focus

- Confirm DR site location and RTO/RPO targets against NCA business-continuity requirements (already open at NCIE-002 Ch.28 §28-review).

Traceability: NCIE-002 Ch.28, Ch.29 §29.9; NCIE-003 Ch.5, Ch.25.

Performance, Capacity, Scalability & Load Engineering

Defines load, capacity, scalability and performance-engineering practices.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.30 — Reference NFR targets (latency, concurrent users, national cell scale) are proposed baselines this stack must be capacity-planned against.

31.1 Performance and Capacity Engineering

Table 30. Performance, capacity and load-engineering technology decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Status
Capacity planning	Sized against NCIE-002 Ch.30 §30.2's reference targets (≥500 concurrent users, national cell scale)	Load-test-driven sizing per service tier, revisited each major release	PROPOSED DESIGN DEFAULT
Horizontal scaling	That stateless services scale out is mandated (Ch.4, NCIE-002 Ch.30); the specific autoscaling metric (request rate/queue depth) is an engineering choice	Orchestrator autoscaling policies keyed to request rate/queue depth	PROPOSED DESIGN DEFAULT
Concurrency controls	Prevents a single tenant/workload from starving others (NCIE-002 Ch.30 §30.3)	Per-tenant rate limiting at the API gateway (Ch.8)	PROPOSED DESIGN DEFAULT
Backpressure/request shedding	Graceful degradation under load rather than uncontrolled failure (NCIE-002 Ch.30 §30.7)	Gateway-level shedding with visible degraded-status signal (Ch.26)	APPROVED UPSTREAM MANDATE

Human Review Focus

- Confirm production-scale targets (concurrent users, national cell count, data volumes) with NCA capacity-planning stakeholders (already open at NCIE-002 Ch.30 §30-review).

Traceability: NCIE-002 Ch.30.

Integration Adapters, RPA & Legacy-System Engineering

Implements replaceable adapters, RPA and legacy-system integration patterns.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.21 — Capability Registry abstracts UiPath/Power Automate/Automation Anywhere and future providers behind one contract; provider neutrality is mandatory. | NCIE-003 Ch.34 — Legacy import passes through the same validation as any source; original identifiers and transformations are retained, never discarded.

32.1 Integration and Legacy Engineering

Table 31. Integration adapters, RPA and legacy-system engineering decisions.

Capability	Engineering Characteristics Required	Candidate/Default	Alternative	Status
Connector framework	Provider-neutral adapter contract (NCIE-002 Ch.21 §21.2 Capability Registry)	Internal adapter interface with one implementation per provider	—	APPROVED UPSTREAM MANDATE
File/SFTP ingestion	Scheduled/triggered acquisition for sources without an API (NCIE-003 Ch.6)	SFTP poller feeding the Ch.6 ingestion gateway	—	PROPOSED DESIGN DEFAULT
RPA	Provider-neutral, behind the Capability Registry; no ARGUS/workflow code depends on one RPA vendor's SDK	UiPath, Power Automate or Automation Anywhere per NCA's existing licensing — selected as a Provider Adapter, not a platform dependency	—	INSTITUTIONAL CONFIRMATION REQUIRED
Anti-corruption layer	Legacy/external system quirks isolated from canonical semantics (NCIE-003 Ch.34)	Adapter-local translation layer; canonical services never see legacy field shapes directly	—	APPROVED UPSTREAM MANDATE
Retries	Idempotent retry with backoff on transient failure (NCIE-003 Ch.6 §6.6 deduplication)	Adapter-level retry policy with idempotency key	—	PROPOSED DESIGN DEFAULT
Reconciliation	Unknown external outcome reconciled, not blindly retried (NCIE-003 Ch.23 §23.2)	Shared reconciliation mechanism with Ch.10's workflow engine	—	APPROVED UPSTREAM MANDATE

Human Review Focus

- Confirm which RPA provider(s) NCA already licenses before approving one as the primary Provider Adapter.
- Confirm which external systems require RPA versus direct API integration (already open at NCIE-003 Ch.23 §23-review).

Traceability: NCIE-002 Ch.21; NCIE-003 Ch.6, Ch.23, Ch.34.

Engineering Governance, Technology Registry & Lifecycle Management

Governs approved technologies, versions, licensing and deprecation.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.29 — Approved technologies require governed version/license/lifecycle tracking, distinct from the choosing record (Ch.35).

33.1 Technology Registry vs Technology Decision Matrix

The Technology Registry (this chapter) governs what has been institutionally approved: version, owner, license, support/security/lifecycle state. The Technology Decision Matrix (Chapter 35) records the choosing process: candidate, rationale, alternative, status. These are independent axes — a Ch.35 status of APPROVED UPSTREAM MANDATE describes where a requirement originated (already dictated by NCIE-001/002/003); it is not the same thing as a Registry Lifecycle State of Approved, which records that NCA has institutionally confirmed the specific technology. A capability can be an Approved Upstream Mandate while its chosen implementation technology is still Proposed — pending approval, exactly as most rows in Chapter 35 now show after the mandate/implementation audit in §33.1A below.

33.1A Mandate/Implementation Separation (Audit Finding)

A prior edition over-classified several implementation mechanisms as APPROVED UPSTREAM MANDATE merely because the capability they served was mandated (e.g. WAL archiving for a mandated PITR capability, Vault Transit for mandated tokenization, a named event-broker topic for a mandated fail-closed audit behaviour). This edition corrects every such row: the status now reflects whether the exact technology/pattern shown — not merely the underlying capability — was itself dictated by NCIE-001/002/003. Where it was not, the row is reclassified PROPOSED DESIGN DEFAULT, and its Engineering Characteristics column states the mandate separately from the implementation. Chapter 35 reflects this corrected classification, extracted directly from the chapter tables so the two never drift apart.

33.2 NCIE Technology Registry (Seeded)

Table 32. Seeded NCIE Technology Registry — representative approved-candidate entries, none yet institutionally approved.

Technology	Purpose	Category	Version Policy	Deployment Scope	Owner	License	Lifecycle State
PostgreSQL	Transactional canonical store	Database	Latest stable major, security-patched	Ch.11 (+ PostGIS/pgvector/Timescale DB extensions, Ch.7/12/16)	NCIE Architecture Function	PostgreSQL (open)	Proposed — pending approval
Kubernetes	Container orchestration	Runtime	N-1 minor version support window	All containerized services (Ch.4)	NCIE Architecture Function / NCA IT Infrastructure	Apache 2.0 (open)	Proposed — pending approval
Apache Kafka	Event streaming/messaging	Integration	LTS releases only	Event broker (Ch.9)	NCIE Architecture Function	Apache 2.0 (open)	Proposed — pending approval
Redis	Cache and distributed state	Data	Latest stable, security-patched	Ch.17	NCIE Architecture Function	BSD/RSA Lv 2 (verify license terms at approval)	Proposed — pending approval
OpenSearch	Search and retrieval	Data	Latest stable major	Ch.14	NCIE Architecture Function	Apache 2.0 (open)	Proposed — pending approval

Technology	Purpose	Category	Version Policy	Deployment Scope	Owner	License	Lifecycle State
Neo4j	Graph technology	Data	Latest stable, community or enterprise per licensing decision	Ch.15	NCIE Architecture Function	GPL (community) / commercial (enterprise) — confirm at approval	Proposed — pending approval
pgvector	Vector retrieval (launch scale)	Data / AI	Tracks PostgreSQL major version	Ch.16	NCIE Architecture Function	PostgreSQL (open)	Proposed — pending approval
MapLibre GL	GIS/map rendering	Visualization	Latest stable	Ch.7	NCIE Architecture Function	BSD (open)	Proposed — pending approval
Keycloak	Identity provider / SSO	Security	Latest stable, security-patched	Ch.23	NCA IT Security	Apache 2.0 (open)	Proposed — pending approval
HashiCorp Vault	Secrets/KMS	Security	Latest stable, security-patched	Ch.24-25	NCA IT Security	BUSL (verify license terms at approval)	Proposed — pending approval
Temporal	Durable workflow engine	Integration	Latest stable	Ch.10	NCIE Architecture Function	MIT (open)	Proposed — pending approval
Prometheus / Grafana	Metrics and dashboards	Observability	Latest stable	Ch.26	NCIE Architecture Function	Apache 2.0 (open)	Proposed — pending approval
OpenTelemetry	Tracing/instrumentation	Observability	Latest stable SDK per language	Ch.26	NCIE Architecture Function	Apache 2.0 (open)	Proposed — pending approval
MinIO	Object/evidence storage	Data	Latest stable, security-patched	Ch.13	NCIE Architecture Function	AGPLv3 (verify license terms at approval)	Proposed — pending approval
React / TypeScript	Frontend framework/type system	Application	Latest stable LTS-equivalent	Ch.6	NCIE Architecture Function	MIT (open)	Proposed — pending approval
RPA Provider (unnamed)	Legacy/portal automation	Integration	Per vendor support policy	Ch.32, behind Capability Registry	NCA Automation Governance	Commercial — depends on NCA's existing licence	Institutional confirmation required

33.2A Version Policy, Not Pinned Versions

Every entry above states an approved version policy or supported-release window (e.g. "N-1 minor version support window", "LTS releases only") rather than a pinned exact version number. Fixing a specific major version before hosting/deployment discovery (Ch.3) is complete would create needless rework; the policy is what this Registry governs, and the exact in-service version is tracked operationally against that policy once deployed.

33.3 Lifecycle, Promotion and Deprecation

PROPOSED DESIGN DEFAULT. Registry promotion invariant: a Technology Registry entry shall not transition to Approved merely because its associated Proposed Design Default has been used in downstream design, prototyping or specification work (see Ch.35 §35.5). Approval requires an explicit institutional decision recorded against the applicable Technology Decision ID, with approving actor and date — the same discipline NCIE-003 Ch.24 requires before a Rule Version is treated as effective. A capability's Ch.35 status of APPROVED UPSTREAM MANDATE never by itself promotes the chosen implementation technology's Registry Lifecycle State.

- Vulnerability status is tracked per entry via Ch.28's SCA scanning; a critical unpatched vulnerability moves Security Status to At Risk and blocks new deployments matching that version policy.
- Deprecation/replacement follows the same change-control discipline as NCIE-003 Ch.1 §1.4 — a superseding entry is added, the prior entry is marked Deprecated with a sunset date, never silently removed.

Human Review Focus

- Confirm license terms (BUSL/AGPL/RSALv2-class technologies above) against NCA procurement/legal policy before approval.

Traceability: NCIE-002 Ch.29.

Reference Deployment Topology & Environment Blueprint

Integrates the selected stack into a logical deployment topology.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 Ch.26 §26.7, Ch.29 — Trust-zone segmentation and environment separation must be visible as a concrete deployment topology, not only a principle.

34.1 Technology Stack Layer Model

Figure 6. Technology Stack Layer Model — how Chapters 4-33 compose into one platform.

01	Edge / API layer (Ch.8: gateway, TLS, rate limiting)
02	Application layer (Ch.5-7, Ch.20-22: backend services, frontend, ARGUS/collaboration)
03	Data layer (Ch.11-17: transactional, analytical, object, search, graph, vector, cache)
04	AI layer (Ch.18-19: Model Gateway, guardrails, evaluation)
05	Integration layer (Ch.9-10, Ch.32: events, workflow, adapters/RPA)
06	Security & observability layer (Ch.23-27: IAM, secrets, DLP, telemetry, SecOps)
07	Management/governance layer (Ch.28, Ch.33: CI/CD, Technology Registry)

34.2 Logical Deployment Topology

Figure 7. Logical deployment topology across NCIE-002 Ch.26's trust zones.

SOURCE	RELATIONSHIP	TARGET
Public/Operator Network	reaches via TLS/API gateway	Edge/API Zone
Edge/API Zone	routes authenticated calls to	Application Zone
Application Zone	reads/writes via governed contracts to	Data Zone
Application Zone	invokes ARGUS via Model Gateway in	AI Zone
AI Zone	never crosses directly into	Data Zone (goes via Application Zone's authorization boundary)
Security/Observability Zone	monitors and gates	Edge, Application, Data, AI and Integration Zones
Management/Governance Zone	deploys and governs	all zones via CI/CD (Ch.28) and the Technology Registry (Ch.33)

34.3 Data Technology Topology

Figure 8. Data technology topology: canonical-vs-derived store relationships (NCIE-003 Ch.32 §32.5).

SOURCE	RELATIONSHIP	TARGET
Transactional Store (Ch. 11, PostgreSQL default)	is the canonical source for	Analytical/Time-Series Store (Ch.12)
Transactional Store	is the canonical source for	Search Index (Ch.14) and Vector Index (Ch.16)
Transactional Store	is the canonical source for	Graph Store (Ch.15)

SOURCE	RELATIONSHIP	TARGET
Object/Evidence Store (Ch.13)	is referenced by pointer from	Transactional Store (never embedded copies)
Cache (Ch.17)	is a derived, rebuildable accelerator over	Transactional Store and Rule/Session state
All derived stores	are rebuildable from and never independently canonical relative to	Transactional Store

Human Review Focus

- Confirm the final zone-to-network-segment mapping with NCA IT Infrastructure once hosting facility (Ch.3) is confirmed.

Traceability: NCIE-002 Ch.26, Ch.29; NCIE-003 Ch.32.

Technology Decision Matrix, Proposed Defaults & Human Review Register

Consolidates proposed defaults, unresolved decisions, rationale and ownership.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 — — Consolidates every technology decision from Chapters 2-32 into one authoritative matrix, per the mandatory-consolidation directive.

35.1 Purpose

Consolidates every technology decision raised in Chapters 2-32 into one Technology Decision Matrix, and every non-mandate decision into one Human Review Register, classified Blocking or Non-Blocking and by reason. Individual chapters are not repeated here beyond a Decision ID and one-line capability/status — the rationale, alternative and engineering characteristics remain in the owning chapter.

35.2 Summary

Table 33. Technology Decision Matrix summary counts.

Metric	Count
Total technology decisions (Ch.2-32)	165
Approved Upstream Mandate	33
Proposed Design Default	115
Institutional Confirmation Required	6
Infrastructure Discovery Required	5
Security/Sovereignty Confirmation Required	6
Blocking (Institutional + Infrastructure + Security)	17
Non-Blocking open (Proposed Design Default)	115

35.3 Technology Decision Matrix

Every technology decision from Chapters 2-32, in chapter order. Full rationale, alternative and engineering characteristics remain in the cited chapter.

Table 34. Complete Technology Decision Matrix (Chapters 2-32).

Decision ID	Ch.	Capability	Proposed/Default	Status
TD-2-1	Ch.2	Code style & linting	Per-language formatter/linter (e.g. Black/Ruff for Python, ESLint/Prettier for TypeScript), enforced in CI (Ch.28)	PROPOSED DESIGN DEFAULT
TD-2-2	Ch.2	API/error conventions	Problem-details-style JSON error envelope; mandatory correlation ID header	PROPOSED DESIGN DEFAULT
TD-2-3	Ch.2	Null/unknown representation	Typed nullable/optional fields plus an explicit status enum	PROPOSED DESIGN DEFAULT
TD-2-4	Ch.2	Versioning	Semantic versioning across services and schemas	PROPOSED DESIGN DEFAULT
TD-3-1	Ch.3	Hosting model	NCA data centre or approved sovereign cloud within Ghana	SECURITY/SOVEREIGNTY CONFIRMATION REQUIRED

Decision ID	Ch.	Capability	Proposed/Default	Status
TD-3-2	Ch.3	Environment tiers	Four-tier environment model, one per NCIE-002 Ch.29	APPROVED UPSTREAM MANDATE
TD-3-3	Ch.3	Trust zones	The 7-zone model in §34.2 (edge/API, application, data, AI, integration, security-observability, management)	PROPOSED DESIGN DEFAULT
TD-3-4	Ch.3	Capacity baseline	Sized once real infrastructure and volumes are confirmed	INFRASTRUCTURE DISCOVERY REQUIRED
TD-4-1	Ch.4	Container runtime	containerd (via Docker build tooling)	PROPOSED DESIGN DEFAULT
TD-4-2	Ch.4	Orchestrator	Kubernetes	PROPOSED DESIGN DEFAULT
TD-4-3	Ch.4	Ingress/networking	NGINX/Envoy-based ingress controller	PROPOSED DESIGN DEFAULT
TD-4-4	Ch.4	Autoscaling	Kubernetes HPA + custom metrics	PROPOSED DESIGN DEFAULT
TD-4-5	Ch.4	Stateful/GPU scheduling	Dedicated node pools with taints/tolerations; GPU device plugin where ARGUS requires local inference	INFRASTRUCTURE DISCOVERY REQUIRED
TD-5-1	Ch.5	Primary backend runtime	Python (type-checked) for domain/AI-adjacent services	INSTITUTIONAL CONFIRMATION REQUIRED
TD-5-2	Ch.5	API framework	FastAPI (Python)	PROPOSED DESIGN DEFAULT
TD-5-3	Ch.5	Background workers	Framework-native worker process consuming the Ch.9 event broker	PROPOSED DESIGN DEFAULT
TD-5-4	Ch.5	Dependency/build tooling	Poetry/pip-tools (Python) or pnpm (Node.js)	PROPOSED DESIGN DEFAULT
TD-5-5	Ch.5	Service template	Internal service template repository	PROPOSED DESIGN DEFAULT
TD-6-1	Ch.6	Web framework	React	PROPOSED DESIGN DEFAULT
TD-6-2	Ch.6	Type system	TypeScript	PROPOSED DESIGN DEFAULT
TD-6-3	Ch.6	State/data layer	React Query / equivalent data-fetching layer	PROPOSED DESIGN DEFAULT
TD-6-4	Ch.6	Routing	Framework-native router	PROPOSED DESIGN DEFAULT
TD-6-5	Ch.6	Build/accessibility tooling	Vite build; automated accessibility linting/testing in CI (Ch.28-29)	PROPOSED DESIGN DEFAULT
TD-7-1	Ch.7	Design system/components	Internal component library on a headless UI kit (e.g. Radix)	PROPOSED DESIGN DEFAULT
TD-7-2	Ch.7	Charting	Recharts/Observable Plot	PROPOSED DESIGN DEFAULT
TD-7-3	Ch.7	GIS/map engine	MapLibre GL (open, avoids per-seat licensing)	INSTITUTIONAL CONFIRMATION REQUIRED
TD-7-4	Ch.7	Spatial services	PostGIS extension on the transactional store (Ch.11)	PROPOSED DESIGN DEFAULT
TD-7-5	Ch.7	Dashboard/export rendering	Server-side headless rendering	PROPOSED DESIGN DEFAULT

Decision ID	Ch.	Capability	Proposed/Default	Status
TD-8-1	Ch.8	Synchronous API protocol	REST/HTTP+JSON	PROPOSED DESIGN DEFAULT
TD-8-2	Ch.8	Internal high-throughput calls	gRPC where justified by measured latency need	PROPOSED DESIGN DEFAULT
TD-8-3	Ch.8	API gateway	Kong / Envoy-based gateway	PROPOSED DESIGN DEFAULT
TD-8-4	Ch.8	Schema tooling	OpenAPI + schema-diff CI gate (Ch.28-29)	PROPOSED DESIGN DEFAULT
TD-8-5	Ch.8	Idempotency enforcement	Idempotency-key middleware in every service	APPROVED UPSTREAM MANDATE
TD-8-6	Ch.8	Rate limiting	Gateway-level rate limiting	PROPOSED DESIGN DEFAULT
TD-9-1	Ch.9	Event broker	Apache Kafka (or Redpanda for lighter ops)	PROPOSED DESIGN DEFAULT
TD-9-2	Ch.9	Queues	Broker-native topics/queues	PROPOSED DESIGN DEFAULT
TD-9-3	Ch.9	Schema registry	Confluent/Karapace-style schema registry	PROPOSED DESIGN DEFAULT
TD-9-4	Ch.9	Dead-letter queue & replay	Broker-native DLQ + replay tooling	PROPOSED DESIGN DEFAULT
TD-9-5	Ch.9	Outbox/inbox pattern	Transactional outbox on the publishing service's own database (Ch.11)	PROPOSED DESIGN DEFAULT
TD-10-1	Ch.10	Workflow/orchestration engine	Temporal	PROPOSED DESIGN DEFAULT
TD-10-2	Ch.10	State machine / BPMN	Engine-native workflow definitions mapped 1:1 to canonical Workflow States	APPROVED UPSTREAM MANDATE
TD-10-3	Ch.10	Timers & waiting dependencies	Engine-native durable timers/signals	PROPOSED DESIGN DEFAULT
TD-10-4	Ch.10	Human tasks & approvals	Engine human-task API integrated with Ch.23 IAM roles	APPROVED UPSTREAM MANDATE
TD-10-5	Ch.10	Reconciliation	Engine-native compensation/reconciliation workflow	PROPOSED DESIGN DEFAULT
TD-11-1	Ch.11	Relational DBMS	PostgreSQL	PROPOSED DESIGN DEFAULT
TD-11-2	Ch.11	HA/replication	Streaming replication + automated failover (e.g. Patroni)	PROPOSED DESIGN DEFAULT
TD-11-3	Ch.11	Migrations	Alembic/Flyway-style migration tooling in CI (Ch.28)	PROPOSED DESIGN DEFAULT
TD-11-4	Ch.11	Backup / PITR	Continuous WAL archiving + scheduled base backups	PROPOSED DESIGN DEFAULT
TD-11-5	Ch.11	Row/column security	Row-level security policies + column encryption (Ch.24-25)	PROPOSED DESIGN DEFAULT
TD-12-1	Ch.12	OLAP/time-series engine	TimescaleDB (PostgreSQL extension, minimises polyglot footprint)	PROPOSED DESIGN DEFAULT
TD-12-2	Ch.12	Warehouse/lakehouse	Deferred until analytical query volume justifies a separate warehouse	INFRASTRUCTURE DISCOVERY REQUIRED

Decision ID	Ch.	Capability	Proposed/Default	Status
TD-12-3	Ch.12	Partitioning	Native partitioning by period + operator	PROPOSED DESIGN DEFAULT
TD-12-4	Ch.12	Materialized views	Scheduled materialized-view refresh	PROPOSED DESIGN DEFAULT
TD-12-5	Ch.12	Historical query	Bitemporal query layer over Ch.11's canonical tables plus this analytical layer	PROPOSED DESIGN DEFAULT
TD-13-1	Ch.13	Object/document storage	MinIO (self-hosted, sovereignty-friendly) or S3-compatible NCA-approved object store	SECURITY/SOVEREIGNTY CONFIRMATION REQUIRED
TD-13-2	Ch.13	Versioning	Object-store native versioning	PROPOSED DESIGN DEFAULT
TD-13-3	Ch.13	WORM / immutability	Object-lock / compliance-mode retention on evidence buckets	PROPOSED DESIGN DEFAULT
TD-13-4	Ch.13	Hashing	SHA-256 computed at registration and re-verified on formal retrieval	PROPOSED DESIGN DEFAULT
TD-13-5	Ch.13	Malware scanning	Scan-on-ingest pipeline ahead of validation (NCIE-003 Ch.6)	PROPOSED DESIGN DEFAULT
TD-14-1	Ch.14	Search engine	OpenSearch (avoids Elastic licensing constraints)	PROPOSED DESIGN DEFAULT
TD-14-2	Ch.14	Facets	Engine-native aggregations	PROPOSED DESIGN DEFAULT
TD-14-3	Ch.14	Authorization filters	Index-per-classification-tier + post-retrieval authorization check	PROPOSED DESIGN DEFAULT
TD-14-4	Ch.14	Snippets	Engine-native highlighting restricted to authorized fields only	PROPOSED DESIGN DEFAULT
TD-14-5	Ch.14	Reindexing	Scheduled/triggered reindex pipeline, never a manual one-off script	PROPOSED DESIGN DEFAULT
TD-15-1	Ch.15	Graph engine	Neo4j	PROPOSED DESIGN DEFAULT
TD-15-2	Ch.15	Query language	Cypher (Neo4j) / openCypher (AGE)	PROPOSED DESIGN DEFAULT
TD-15-3	Ch.15	Temporal edges	Edge properties carrying effective-from/to, queried via Ch.5's bitemporal pattern	APPROVED UPSTREAM MANDATE
TD-15-4	Ch.15	Access filtering	Query-time authorization filter preventing traversal into unauthorized-classification nodes	SECURITY/SOVEREIGNTY CONFIRMATION REQUIRED
TD-15-5	Ch.15	Graph analytics	Engine-native graph algorithms (community detection, shortest-path) as analyst-invoked queries, never autonomous	PROPOSED DESIGN DEFAULT
TD-16-1	Ch.16	Embedding service	Embedding models registered in the Approved Model Registry (Ch. 18)	APPROVED UPSTREAM MANDATE
TD-16-2	Ch.16	Vector store	pgvector (PostgreSQL extension) for launch scale	PROPOSED DESIGN DEFAULT

Decision ID	Ch.	Capability	Proposed/Default	Status
TD-16-3	Ch.16	Metadata filters	Filtered ANN search restricted to the querying identity's authorized scope	APPROVED UPSTREAM MANDATE
TD-16-4	Ch.16	Index isolation	Separate vector index per classification tier	SECURITY/SOVEREIGNTY CONFIRMATION REQUIRED
TD-16-5	Ch.16	Re-embedding	Scheduled, audited re-embedding pipeline	PROPOSED DESIGN DEFAULT
TD-16-6	Ch.16	Evaluation	Retrieval-quality evaluation harness integrated with Ch.19's AI evaluation suite	PROPOSED DESIGN DEFAULT
TD-17-1	Ch.17	Distributed cache	Redis	PROPOSED DESIGN DEFAULT
TD-17-2	Ch.17	Session/ephemeral state	Redis-backed session store	PROPOSED DESIGN DEFAULT
TD-17-3	Ch.17	Cache invalidation	Event-driven invalidation on permission/rule change (Ch.9 event broker)	PROPOSED DESIGN DEFAULT
TD-18-1	Ch.18	Model Gateway	Internal gateway service, provider-neutral by construction	APPROVED UPSTREAM MANDATE
TD-18-2	Ch.18	Provider adapters	Adapter-per-provider pattern behind a common interface	APPROVED UPSTREAM MANDATE
TD-18-3	Ch.18	Approved Model Registry	Registry table in the transactional store (Ch.11), administered via Ch.33	PROPOSED DESIGN DEFAULT
TD-18-4	Ch.18	Routing policy engine	Policy evaluated per-request before model selection	APPROVED UPSTREAM MANDATE
TD-18-5	Ch.18	Context assembly	Context Builder service filtering by requester's effective authorization	APPROVED UPSTREAM MANDATE
TD-18-6	Ch.18	Fallback	Routing policy defines an approved fallback chain per task type	PROPOSED DESIGN DEFAULT
TD-19-1	Ch.19	Grounding controls	Grounding Layer enforced in the Model Gateway response path (Ch.18)	APPROVED UPSTREAM MANDATE
TD-19-2	Ch.19	Injection defenses	Input-classification filter tagging untrusted content before it reaches a prompt	APPROVED UPSTREAM MANDATE
TD-19-3	Ch.19	Evaluation harness	Offline eval suite (context tracking, attribution, thread separation, restraint) run per model/provider	APPROVED UPSTREAM MANDATE
TD-19-4	Ch.19	Red-team testing	Scheduled red-team exercises ahead of each model/provider approval (Ch.33 registry gate)	PROPOSED DESIGN DEFAULT
TD-19-5	Ch.19	Regression gates	CI gate blocking Model Registry promotion on failed evaluation suite (Ch.28-29)	PROPOSED DESIGN DEFAULT
TD-19-6	Ch.19	Abstention	Grounding Layer returns an explicit "cannot verify" response class	APPROVED UPSTREAM MANDATE
TD-20-1	Ch.20	Tool Registry	Internal Tool Registry service, versioned (Ch.33)	APPROVED UPSTREAM MANDATE

Decision ID	Ch.	Capability	Proposed/Default	Status
TD-20-2	Ch.20	Tool Gateway enforcement	Gateway middleware enforcing the full checklist in §20.3	APPROVED UPSTREAM MANDATE
TD-20-3	Ch.20	Agent orchestrator	Workflow-engine-hosted agent tasks (Ch.10), not a separate autonomous runtime	PROPOSED DESIGN DEFAULT
TD-20-4	Ch.20	Sandboxing	Per-invocation isolated execution context (container/process, Ch.4)	PROPOSED DESIGN DEFAULT
TD-20-5	Ch.20	Execution receipts	Receipt written to Ch.27 (Audit) on every tool execution	APPROVED UPSTREAM MANDATE
TD-21-1	Ch.21	Realtime transport	WebSocket gateway (e.g. Socket.IO/native WS) behind the API gateway (Ch.8)	PROPOSED DESIGN DEFAULT
TD-21-2	Ch.21	Presence	Presence service backed by cache (Ch.17)	PROPOSED DESIGN DEFAULT
TD-21-3	Ch.21	Collaborative editing	CRDT-based or operational-transform sync for shared documents/boards	PROPOSED DESIGN DEFAULT
TD-21-4	Ch.21	Whiteboard/evidence board	Board component referencing Evidence Objects by ID only (Ch.13)	APPROVED UPSTREAM MANDATE
TD-21-5	Ch.21	Room events	Room event bus on top of Ch.9's event broker	APPROVED UPSTREAM MANDATE
TD-21-6	Ch.21	ARGUS hooks	Room event subscription feeding the Proactive Contribution Controller (Ch.18/20)	APPROVED UPSTREAM MANDATE
TD-22-1	Ch.22	Memory service	Service backed by the transactional store (Ch.11), not provider-side memory features	PROPOSED DESIGN DEFAULT
TD-22-2	Ch.22	Knowledge repository	Same transactional store, dedicated schema (Ch.11)	PROPOSED DESIGN DEFAULT
TD-22-3	Ch.22	Dependency/context graph	Reconciliation job reusing Ch.15's graph technology where relational joins are insufficient	PROPOSED DESIGN DEFAULT
TD-22-4	Ch.22	Snapshots	Reuses Ch.12's historical/bitemporal query layer	PROPOSED DESIGN DEFAULT
TD-22-5	Ch.22	Retrieval	Retrieval layer enforcing Ch.16's classification-aware filtering	APPROVED UPSTREAM MANDATE
TD-23-1	Ch.23	IdP/SSO	Keycloak	INSTITUTIONAL CONFIRMATION REQUIRED
TD-23-2	Ch.23	MFA	TOTP authenticator app; SMS fallback disabled by default given SIM-fraud relevance	PROPOSED DESIGN DEFAULT
TD-23-3	Ch.23	RBAC/ABAC/PBAC	Policy engine (e.g. OPA) fronting the four-layer authorization model	PROPOSED DESIGN DEFAULT
TD-23-4	Ch.23	Workload identity	SPIFFE/SPIRE-style workload identity or IdP-issued service accounts	PROPOSED DESIGN DEFAULT

Decision ID	Ch.	Capability	Proposed/Default	Status
TD-23-5	Ch.23	Privileged access management (PAM)	PAM module on top of the IdP with mandatory justification capture	PROPOSED DESIGN DEFAULT
TD-23-6	Ch.23	Federation	IdP-federated SSO across the service mesh (Ch.8)	PROPOSED DESIGN DEFAULT
TD-24-1	Ch.24	Secrets vault	HashiCorp Vault	PROPOSED DESIGN DEFAULT
TD-24-2	Ch.24	KMS/HSM	Vault Transit engine or dedicated HSM where sovereignty/compliance requires hardware-backed keys	SECURITY/SOVEREIGNTY CONFIRMATION REQUIRED
TD-24-3	Ch.24	PKI / certificate automation	cert-manager (Kubernetes-native) with an internal or NCA-approved CA	PROPOSED DESIGN DEFAULT
TD-24-4	Ch.24	Rotation	Automated rotation policies in the vault/cert-manager	PROPOSED DESIGN DEFAULT
TD-24-5	Ch.24	Encryption/signing	Application-layer envelope encryption using KMS-issued data keys	PROPOSED DESIGN DEFAULT
TD-25-1	Ch.25	Tokenization	Vault Transit tokenization (Ch.24) or a dedicated tokenization service	PROPOSED DESIGN DEFAULT
TD-25-2	Ch.25	Masking	Server-side masking applied before response serialization, never client-side only	APPROVED UPSTREAM MANDATE
TD-25-3	Ch.25	Field encryption	Application-layer field encryption using KMS-issued keys	PROPOSED DESIGN DEFAULT
TD-25-4	Ch.25	DLP	DLP policy on export/API-response paths flagging bulk protected-field extraction	SECURITY/SOVEREIGNTY CONFIRMATION REQUIRED
TD-25-5	Ch.25	Protected reveal	Protected-reveal endpoint requiring explicit authorization check + audit event (Ch.27)	APPROVED UPSTREAM MANDATE
TD-25-6	Ch.25	Export controls	Export gateway enforcing classification + Ch.27 audit on every export	APPROVED UPSTREAM MANDATE
TD-26-1	Ch.26	Logs	Structured JSON logging shipped to a central log store	PROPOSED DESIGN DEFAULT
TD-26-2	Ch.26	Metrics	Prometheus-compatible metrics	PROPOSED DESIGN DEFAULT
TD-26-3	Ch.26	Traces	OpenTelemetry instrumentation	PROPOSED DESIGN DEFAULT
TD-26-4	Ch.26	APM/dashboards	Grafana dashboards, one board family per category	PROPOSED DESIGN DEFAULT
TD-26-5	Ch.26	Alerting	Alertmanager routing rules keyed to signal category and severity	PROPOSED DESIGN DEFAULT
TD-26-6	Ch.26	SLOs	SLO burn-rate alerting on the reference targets	PROPOSED DESIGN DEFAULT
TD-27-1	Ch.27	SIEM	Wazuh (open) or NCA-existing SIEM	INSTITUTIONAL CONFIRMATION REQUIRED

Decision ID	Ch.	Capability	Proposed/Default	Status
TD-27-2	Ch.27	SOAR	SIEM-integrated SOAR module where the chosen SIEM supports it	PROPOSED DESIGN DEFAULT
TD-27-3	Ch.27	WAF/IDS/IPS	WAF at the API gateway (Ch.8); network IDS/IPS at zone boundaries	PROPOSED DESIGN DEFAULT
TD-27-4	Ch.27	EDR integration	EDR agent feeding SIEM	INFRASTRUCTURE DISCOVERY REQUIRED
TD-27-5	Ch.27	Security-event pipeline	Same event broker (Ch.9) carrying a dedicated security-event topic into the SIEM	PROPOSED DESIGN DEFAULT
TD-28-1	Ch.28	Source control	Git (GitLab/GitHub, self-hosted or NCA-approved)	INSTITUTIONAL CONFIRMATION REQUIRED
TD-28-2	Ch.28	CI/CD	GitLab CI or GitHub Actions, self-hosted runners in the sovereign environment (Ch.3)	PROPOSED DESIGN DEFAULT
TD-28-3	Ch.28	Artifact/container registry	Self-hosted registry (e.g. Harbor) inside the sovereign environment	PROPOSED DESIGN DEFAULT
TD-28-4	Ch.28	SAST/SCA/secret scanning	Integrated scanners in the CI pipeline (e.g. Semgrep, Trivy, gitleaks)	PROPOSED DESIGN DEFAULT
TD-28-5	Ch.28	IaC	Terraform / Kubernetes manifests under source control	PROPOSED DESIGN DEFAULT
TD-28-6	Ch.28	Policy-as-code	OPA/Conftest policy gates in CI	PROPOSED DESIGN DEFAULT
TD-28-7	Ch.28	SBOM	Generated per release build (e.g. Syft)	PROPOSED DESIGN DEFAULT
TD-29-1	Ch.29	Unit/integration	Language-native test frameworks (pytest/Jest) in CI (Ch.28)	PROPOSED DESIGN DEFAULT
TD-29-2	Ch.29	Contract testing	Consumer-driven contract tests (e.g. Pact) or schema-diff gate	PROPOSED DESIGN DEFAULT
TD-29-3	Ch.29	End-to-end/UI automation	Playwright	PROPOSED DESIGN DEFAULT
TD-29-4	Ch.29	Performance testing	k6 or similar load-testing tool in a staging environment	PROPOSED DESIGN DEFAULT
TD-29-5	Ch.29	Security testing	DAST tool integrated pre-release; periodic external pen-test	PROPOSED DESIGN DEFAULT
TD-29-6	Ch.29	Data testing	Test harness asserting bitemporal correctness and field-level authorization per entity family	APPROVED UPSTREAM MANDATE
TD-29-7	Ch.29	Workflow/resilience testing	Chaos/failure-injection testing in staging	PROPOSED DESIGN DEFAULT
TD-29-8	Ch.29	AI evaluation	Offline evaluation harness gating Model Registry promotion (Ch.33)	APPROVED UPSTREAM MANDATE
TD-30-1	Ch.30	HA/replication	Multi-replica deployment per service tier	PROPOSED DESIGN DEFAULT

Decision ID	Ch.	Capability	Proposed/Default	Status
TD-30-2	Ch.30	Backup	Automated scheduled backups per store (Ch.11-17)	APPROVED UPSTREAM MANDATE
TD-30-3	Ch.30	PITR	Continuous WAL archiving (Ch.11) + object-store versioning (Ch.13)	PROPOSED DESIGN DEFAULT
TD-30-4	Ch.30	Cross-site DR	Warm-standby DR site within Ghanaian jurisdiction (Ch.3 sovereignty posture)	INFRASTRUCTURE DISCOVERY REQUIRED
TD-30-5	Ch.30	Failover	Orchestrator-driven failover (Ch.4) + database failover (Ch.11)	PROPOSED DESIGN DEFAULT
TD-30-6	Ch.30	Restore testing	Scheduled DR/restore drills with recorded results	APPROVED UPSTREAM MANDATE
TD-31-1	Ch.31	Capacity planning	Load-test-driven sizing per service tier, revisited each major release	PROPOSED DESIGN DEFAULT
TD-31-2	Ch.31	Horizontal scaling	Orchestrator autoscaling policies keyed to request rate/queue depth	PROPOSED DESIGN DEFAULT
TD-31-3	Ch.31	Concurrency controls	Per-tenant rate limiting at the API gateway (Ch.8)	PROPOSED DESIGN DEFAULT
TD-31-4	Ch.31	Backpressure/request shedding	Gateway-level shedding with visible degraded-status signal (Ch.26)	APPROVED UPSTREAM MANDATE
TD-32-1	Ch.32	Connector framework	Internal adapter interface with one implementation per provider	APPROVED UPSTREAM MANDATE
TD-32-2	Ch.32	File/SFTP ingestion	SFTP poller feeding the Ch.6 ingestion gateway	PROPOSED DESIGN DEFAULT
TD-32-3	Ch.32	RPA	UiPath, Power Automate or Automation Anywhere per NCA's existing licensing — selected as a Provider Adapter, not a platform dependency	INSTITUTIONAL CONFIRMATION REQUIRED
TD-32-4	Ch.32	Anti-corruption layer	Adapter-local translation layer; canonical services never see legacy field shapes directly	APPROVED UPSTREAM MANDATE
TD-32-5	Ch.32	Retries	Adapter-level retry policy with idempotency key	PROPOSED DESIGN DEFAULT
TD-32-6	Ch.32	Reconciliation	Shared reconciliation mechanism with Ch.10's workflow engine	APPROVED UPSTREAM MANDATE

35.4 Human Review Register

Every decision not yet an Approved Upstream Mandate, classified Blocking (requires resolution before that technology can be treated as settled) or Non-Blocking (a defensible default that may proceed provisionally) and by reason.

Table 35. Human Review Register: all non-mandate technology decisions.

Decision ID	Ch.	Capability	Reason	Blocking?
TD-2-1	Ch.2	Code style & linting	Proposed Design Default	NON-BLOCKING
TD-2-2	Ch.2	API/error conventions	Proposed Design Default	NON-BLOCKING

Decision ID	Ch.	Capability	Reason	Blocking?
TD-2-3	Ch.2	Null/unknown representation	Proposed Design Default	NON-BLOCKING
TD-2-4	Ch.2	Versioning	Proposed Design Default	NON-BLOCKING
TD-3-1	Ch.3	Hosting model	Security/Sovereignty	BLOCKING
TD-3-3	Ch.3	Trust zones	Proposed Design Default	NON-BLOCKING
TD-3-4	Ch.3	Capacity baseline	Infrastructure Discovery	BLOCKING
TD-4-1	Ch.4	Container runtime	Proposed Design Default	NON-BLOCKING
TD-4-2	Ch.4	Orchestrator	Proposed Design Default	NON-BLOCKING
TD-4-3	Ch.4	Ingress/networking	Proposed Design Default	NON-BLOCKING
TD-4-4	Ch.4	Autoscaling	Proposed Design Default	NON-BLOCKING
TD-4-5	Ch.4	Stateful/GPU scheduling	Infrastructure Discovery	BLOCKING
TD-5-1	Ch.5	Primary backend runtime	Institutional	BLOCKING
TD-5-2	Ch.5	API framework	Proposed Design Default	NON-BLOCKING
TD-5-3	Ch.5	Background workers	Proposed Design Default	NON-BLOCKING
TD-5-4	Ch.5	Dependency/build tooling	Proposed Design Default	NON-BLOCKING
TD-5-5	Ch.5	Service template	Proposed Design Default	NON-BLOCKING
TD-6-1	Ch.6	Web framework	Proposed Design Default	NON-BLOCKING
TD-6-2	Ch.6	Type system	Proposed Design Default	NON-BLOCKING
TD-6-3	Ch.6	State/data layer	Proposed Design Default	NON-BLOCKING
TD-6-4	Ch.6	Routing	Proposed Design Default	NON-BLOCKING
TD-6-5	Ch.6	Build/accessibility tooling	Proposed Design Default	NON-BLOCKING
TD-7-1	Ch.7	Design system/components	Proposed Design Default	NON-BLOCKING
TD-7-2	Ch.7	Charting	Proposed Design Default	NON-BLOCKING
TD-7-3	Ch.7	GIS/map engine	Institutional	BLOCKING
TD-7-4	Ch.7	Spatial services	Proposed Design Default	NON-BLOCKING
TD-7-5	Ch.7	Dashboard/export rendering	Proposed Design Default	NON-BLOCKING
TD-8-1	Ch.8	Synchronous API protocol	Proposed Design Default	NON-BLOCKING
TD-8-2	Ch.8	Internal high-throughput calls	Proposed Design Default	NON-BLOCKING
TD-8-3	Ch.8	API gateway	Proposed Design Default	NON-BLOCKING
TD-8-4	Ch.8	Schema tooling	Proposed Design Default	NON-BLOCKING
TD-8-6	Ch.8	Rate limiting	Proposed Design Default	NON-BLOCKING
TD-9-1	Ch.9	Event broker	Proposed Design Default	NON-BLOCKING
TD-9-2	Ch.9	Queues	Proposed Design Default	NON-BLOCKING
TD-9-3	Ch.9	Schema registry	Proposed Design Default	NON-BLOCKING
TD-9-4	Ch.9	Dead-letter queue & replay	Proposed Design Default	NON-BLOCKING
TD-9-5	Ch.9	Outbox/inbox pattern	Proposed Design Default	NON-BLOCKING
TD-10-1	Ch.10	Workflow/orchestration engine	Proposed Design Default	NON-BLOCKING
TD-10-3	Ch.10	Timers & waiting dependencies	Proposed Design Default	NON-BLOCKING
TD-10-5	Ch.10	Reconciliation	Proposed Design Default	NON-BLOCKING

Decision ID	Ch.	Capability	Reason	Blocking?
TD-11-1	Ch.11	Relational DBMS	Proposed Design Default	NON-BLOCKING
TD-11-2	Ch.11	HA/replication	Proposed Design Default	NON-BLOCKING
TD-11-3	Ch.11	Migrations	Proposed Design Default	NON-BLOCKING
TD-11-4	Ch.11	Backup / PITR	Proposed Design Default	NON-BLOCKING
TD-11-5	Ch.11	Row/column security	Proposed Design Default	NON-BLOCKING
TD-12-1	Ch.12	OLAP/time-series engine	Proposed Design Default	NON-BLOCKING
TD-12-2	Ch.12	Warehouse/lakehouse	Infrastructure Discovery	BLOCKING
TD-12-3	Ch.12	Partitioning	Proposed Design Default	NON-BLOCKING
TD-12-4	Ch.12	Materialized views	Proposed Design Default	NON-BLOCKING
TD-12-5	Ch.12	Historical query	Proposed Design Default	NON-BLOCKING
TD-13-1	Ch.13	Object/document storage	Security/Sovereignty	BLOCKING
TD-13-2	Ch.13	Versioning	Proposed Design Default	NON-BLOCKING
TD-13-3	Ch.13	WORM / immutability	Proposed Design Default	NON-BLOCKING
TD-13-4	Ch.13	Hashing	Proposed Design Default	NON-BLOCKING
TD-13-5	Ch.13	Malware scanning	Proposed Design Default	NON-BLOCKING
TD-14-1	Ch.14	Search engine	Proposed Design Default	NON-BLOCKING
TD-14-2	Ch.14	Facets	Proposed Design Default	NON-BLOCKING
TD-14-3	Ch.14	Authorization filters	Proposed Design Default	NON-BLOCKING
TD-14-4	Ch.14	Snippets	Proposed Design Default	NON-BLOCKING
TD-14-5	Ch.14	Reindexing	Proposed Design Default	NON-BLOCKING
TD-15-1	Ch.15	Graph engine	Proposed Design Default	NON-BLOCKING
TD-15-2	Ch.15	Query language	Proposed Design Default	NON-BLOCKING
TD-15-4	Ch.15	Access filtering	Security/Sovereignty	BLOCKING
TD-15-5	Ch.15	Graph analytics	Proposed Design Default	NON-BLOCKING
TD-16-2	Ch.16	Vector store	Proposed Design Default	NON-BLOCKING
TD-16-4	Ch.16	Index isolation	Security/Sovereignty	BLOCKING
TD-16-5	Ch.16	Re-embedding	Proposed Design Default	NON-BLOCKING
TD-16-6	Ch.16	Evaluation	Proposed Design Default	NON-BLOCKING
TD-17-1	Ch.17	Distributed cache	Proposed Design Default	NON-BLOCKING
TD-17-2	Ch.17	Session/ephemeral state	Proposed Design Default	NON-BLOCKING
TD-17-3	Ch.17	Cache invalidation	Proposed Design Default	NON-BLOCKING
TD-18-3	Ch.18	Approved Model Registry	Proposed Design Default	NON-BLOCKING
TD-18-6	Ch.18	Fallback	Proposed Design Default	NON-BLOCKING
TD-19-4	Ch.19	Red-team testing	Proposed Design Default	NON-BLOCKING
TD-19-5	Ch.19	Regression gates	Proposed Design Default	NON-BLOCKING
TD-20-3	Ch.20	Agent orchestrator	Proposed Design Default	NON-BLOCKING
TD-20-4	Ch.20	Sandboxing	Proposed Design Default	NON-BLOCKING
TD-21-1	Ch.21	Realtime transport	Proposed Design Default	NON-BLOCKING
TD-21-2	Ch.21	Presence	Proposed Design Default	NON-BLOCKING
TD-21-3	Ch.21	Collaborative editing	Proposed Design Default	NON-BLOCKING
TD-22-1	Ch.22	Memory service	Proposed Design Default	NON-BLOCKING
TD-22-2	Ch.22	Knowledge repository	Proposed Design Default	NON-BLOCKING

Decision ID	Ch.	Capability	Reason	Blocking?
TD-22-3	Ch.22	Dependency/context graph	Proposed Design Default	NON-BLOCKING
TD-22-4	Ch.22	Snapshots	Proposed Design Default	NON-BLOCKING
TD-23-1	Ch.23	IdP/SSO	Institutional	BLOCKING
TD-23-2	Ch.23	MFA	Proposed Design Default	NON-BLOCKING
TD-23-3	Ch.23	RBAC/ABAC/PBAC	Proposed Design Default	NON-BLOCKING
TD-23-4	Ch.23	Workload identity	Proposed Design Default	NON-BLOCKING
TD-23-5	Ch.23	Privileged access management (PAM)	Proposed Design Default	NON-BLOCKING
TD-23-6	Ch.23	Federation	Proposed Design Default	NON-BLOCKING
TD-24-1	Ch.24	Secrets vault	Proposed Design Default	NON-BLOCKING
TD-24-2	Ch.24	KMS/HSM	Security/Sovereignty	BLOCKING
TD-24-3	Ch.24	PKI / certificate automation	Proposed Design Default	NON-BLOCKING
TD-24-4	Ch.24	Rotation	Proposed Design Default	NON-BLOCKING
TD-24-5	Ch.24	Encryption/signing	Proposed Design Default	NON-BLOCKING
TD-25-1	Ch.25	Tokenization	Proposed Design Default	NON-BLOCKING
TD-25-3	Ch.25	Field encryption	Proposed Design Default	NON-BLOCKING
TD-25-4	Ch.25	DLP	Security/Sovereignty	BLOCKING
TD-26-1	Ch.26	Logs	Proposed Design Default	NON-BLOCKING
TD-26-2	Ch.26	Metrics	Proposed Design Default	NON-BLOCKING
TD-26-3	Ch.26	Traces	Proposed Design Default	NON-BLOCKING
TD-26-4	Ch.26	APM/dashboards	Proposed Design Default	NON-BLOCKING
TD-26-5	Ch.26	Alerting	Proposed Design Default	NON-BLOCKING
TD-26-6	Ch.26	SLOs	Proposed Design Default	NON-BLOCKING
TD-27-1	Ch.27	SIEM	Institutional	BLOCKING
TD-27-2	Ch.27	SOAR	Proposed Design Default	NON-BLOCKING
TD-27-3	Ch.27	WAF/IDS/IPS	Proposed Design Default	NON-BLOCKING
TD-27-4	Ch.27	EDR integration	Infrastructure Discovery	BLOCKING
TD-27-5	Ch.27	Security-event pipeline	Proposed Design Default	NON-BLOCKING
TD-28-1	Ch.28	Source control	Institutional	BLOCKING
TD-28-2	Ch.28	CI/CD	Proposed Design Default	NON-BLOCKING
TD-28-3	Ch.28	Artifact/container registry	Proposed Design Default	NON-BLOCKING
TD-28-4	Ch.28	SAST/SCA/secret scanning	Proposed Design Default	NON-BLOCKING
TD-28-5	Ch.28	IaC	Proposed Design Default	NON-BLOCKING
TD-28-6	Ch.28	Policy-as-code	Proposed Design Default	NON-BLOCKING
TD-28-7	Ch.28	SBOM	Proposed Design Default	NON-BLOCKING
TD-29-1	Ch.29	Unit/integration	Proposed Design Default	NON-BLOCKING
TD-29-2	Ch.29	Contract testing	Proposed Design Default	NON-BLOCKING
TD-29-3	Ch.29	End-to-end/UI automation	Proposed Design Default	NON-BLOCKING
TD-29-4	Ch.29	Performance testing	Proposed Design Default	NON-BLOCKING
TD-29-5	Ch.29	Security testing	Proposed Design Default	NON-BLOCKING

Decision ID	Ch.	Capability	Reason	Blocking?
TD-29-7	Ch.29	Workflow/resilience testing	Proposed Design Default	NON-BLOCKING
TD-30-1	Ch.30	HA/replication	Proposed Design Default	NON-BLOCKING
TD-30-3	Ch.30	PITR	Proposed Design Default	NON-BLOCKING
TD-30-4	Ch.30	Cross-site DR	Infrastructure Discovery	BLOCKING
TD-30-5	Ch.30	Failover	Proposed Design Default	NON-BLOCKING
TD-31-1	Ch.31	Capacity planning	Proposed Design Default	NON-BLOCKING
TD-31-2	Ch.31	Horizontal scaling	Proposed Design Default	NON-BLOCKING
TD-31-3	Ch.31	Concurrency controls	Proposed Design Default	NON-BLOCKING
TD-32-2	Ch.32	File/SFTP ingestion	Proposed Design Default	NON-BLOCKING
TD-32-3	Ch.32	RPA	Institutional	BLOCKING
TD-32-5	Ch.32	Retries	Proposed Design Default	NON-BLOCKING

35.5 Prioritisation and the Limits of a Non-Blocking Default

The 17 Blocking items (Institutional/Infrastructure/Security) should be resolved before their affected chapters' technology can move to Approved status.

PROPOSED DESIGN DEFAULT. Non-Blocking Proposed Design Defaults (115 items) may be used provisionally for downstream design continuation, prototyping, proof-of-concept work, technical elaboration and specification development in NCIE-005 through NCIE-016. They shall not be interpreted as procurement approval, production authorization or institutional technology approval. Moving a Non-Blocking item from Proposed to Registry Lifecycle State Approved (Ch.33 §33.3) still requires an explicit institutional decision against its Decision ID — non-blocking status affects what may proceed provisionally, not what has been approved.

Traceability: Consolidates Chapters 2-32; individual chapters trace to NCIE-002/003 as cited therein.

NCIE-004 Acceptance, Traceability & Handover Gate

Defines traceability, acceptance and controlled handover of NCIE-004.

Status: IN DEVELOPMENT — FOR HUMAN REVIEW.

Upstream: NCIE-002 — — Formal closure of the authority chain: NCIE-001 requirements → NCIE-002 architecture → NCIE-003 canonical data → NCIE-004 technology.

36.1 Traceability Matrix

Table 36. NCIE-004 chapter-to-upstream traceability matrix.

Ch.	Title	Primary Upstream Dependency
1	Engineering Mandate & Selection Doctrine	NCIE-002 Ch.1
2	Engineering Principles & Standards	NCIE-003 Ch.2, Ch.30
3	Target Runtime, Hosting & Environment	NCIE-002 Ch.1 §1.6, Ch.26, Ch.28, Ch.30; NCIE-003 Ch.32, Ch.35-36
4	Containerization & Orchestration	NCIE-002 Ch.4 §4.2, Ch.29, Ch.30
5	Backend Application Stack	NCIE-002 Ch.4, Ch.5, Ch.25
6	Frontend Web Application Stack	NCIE-002 Ch.3, Ch.30 §30.4
7	Design System, Visualization & GIS	NCIE-002 Ch.3, Ch.8, Ch.30 §30.4; NCIE-003 Ch.8
8	API Gateway & Service Communication	NCIE-002 Ch.4, Ch.25; NCIE-003 Ch.33
9	Event Streaming & Messaging	NCIE-002 Ch.25; NCIE-003 Ch.23, Ch.28
10	Workflow, BPM & Process Engine	NCIE-002 Ch.21; NCIE-003 Ch.23
11	Transactional Database	NCIE-002 Ch.5; NCIE-003 Ch.5, Ch.24, Ch.32
12	Analytical/Time-Series Platform	NCIE-002 Ch.30; NCIE-003 Ch.7, Ch.11, Ch.32, Ch.35
13	Object/Document/Evidence Storage	NCIE-003 Ch.6, Ch.17, Ch.31, Ch.35
14	Search, Indexing & Retrieval	NCIE-003 Ch.4 §4.89-4.93, Ch.36
15	Graph Technology	NCIE-003 Ch.5, Ch.19
16	Vector Retrieval & Semantic Search	NCIE-002 Ch.26 §26.3; NCIE-003 Ch.36 §36.4
17	Cache & Distributed State	NCIE-002 Ch.4, Ch.30 §30.5
18	ARGUS AI Platform & Model Gateway	NCIE-002 Ch.19 §19.11, Ch.24; NCIE-003 Ch.20
19	AI Guardrails, Evaluation & Safety	NCIE-002 Ch.19, Ch.24
20	Agentic Orchestration & Tool Gateway	NCIE-002 Ch.19, Ch.21; NCIE-003 Ch.20, Ch.23
21	Collaborative Intelligence Stack	NCIE-002 Ch.19, Ch.20; NCIE-003 Ch.20-21, Ch.27
22	Memory, Knowledge & Context Stack	NCIE-002 Ch.20 §20.2, §20.12; NCIE-003 Ch.22
23	Identity, Authentication & Access	NCIE-002 Ch.4, Ch.26; NCIE-003 Ch.26
24	Secrets, Keys, Certificates & Crypto	NCIE-002 Ch.26, Ch.29 §29.6; NCIE-003 Ch.14 §14.6, Ch.36
25	Data Protection, DLP & Masking	NCIE-002 Ch.26; NCIE-003 Ch.14, Ch.31, Ch.36
26	Observability & Telemetry	NCIE-002 Ch.27; NCIE-003 Ch.29, Ch.37
27	Security Operations & SIEM	NCIE-002 Ch.26; NCIE-003 Ch.27

Ch.	Title	Primary Upstream Dependency
28	DevSecOps & CI/CD Toolchain	NCIE-002 Ch.29
29	Testing & Quality Engineering	NCIE-002 Ch.31; NCIE-003 Ch.38
30	Resilience, Backup & DR	NCIE-002 Ch.28, Ch.29 §29.9; NCIE-003 Ch.5, Ch.25
31	Performance & Capacity Engineering	NCIE-002 Ch.30
32	Integration Adapters, RPA & Legacy	NCIE-002 Ch.21; NCIE-003 Ch.6, Ch.23, Ch.34
33	Technology Registry & Lifecycle	NCIE-002 Ch.29
34	Reference Deployment Topology	NCIE-002 Ch.26 §26.7, Ch.29; NCIE-003 Ch.32
35	Technology Decision Matrix & Review Register	Consolidates Ch.2-32
36	Acceptance, Traceability & Handover	Closes NCIE-001→002→003→004 authority chain

36.2 Acceptance Checklist

- No material contradiction with NCIE-002/003 remains unresolved (none flagged STRUCTURAL CHANGE REQUIRED in this edition, per Ch.1 §1.2).
- Every technology decision in Ch.35's matrix carries one of the five statuses in front matter §0.3 — none is asserted as settled fact beyond its actual status.
- The Technology Registry (Ch.33) and Decision Matrix (Ch.35) agree — every Registry entry traces to a Ch.35 Decision ID.
- No named product anywhere in this document is asserted as "NCIE uses X" beyond Approved Upstream Mandate status.
- Provider neutrality is preserved for AI models, cloud, databases, search, vector stores, graph technology, workflow engines, message brokers and external integration providers (Ch.9-10, Ch.14-18, Ch.32).
- ARGUS engineering (Ch.18) never architects directly around one provider SDK.
- The AI outage boundary (Ch.18 §18.3, Ch.19) is preserved: deterministic NCIE operation continues without ARGUS.
- Ghana Card/Passport and Anti-Fraud location protections (Ch.25) apply consistently across every chapter that touches them.
- The surcharge rate and SIM registration limits remain governed Rule/Parameter values (NCIE-003 Ch.24), never hard-coded anywhere in this stack.

36.3 Technology Registry Snapshot

This gate references Chapter 33's Registry as of this edition: 16 seeded technologies, all at Lifecycle State "Proposed — pending approval" or "Institutional confirmation required" — none yet Approved. No entry advances without an institutional decision recorded against its Chapter 35 Decision ID.

36.4 Handover

On acceptance, this technology baseline hands forward to NCIE-005 (ARGUS), NCIE-006 (Memory), NCIE-007 (AI/Agent Orchestration), NCIE-009 (Security Architecture), NCIE-011 (API/Integration), NCIE-012 (UX/UI), NCIE-014 (Database & Storage Engineering), NCIE-015 (DevSecOps/Infrastructure) and NCIE-016 (Testing) — each of which implements this document's technology families in detail without re-deciding them, per Ch.1 §1.3's boundary.

Human Review Focus

- Confirm NCIE-004 approvers and their scope of authority.
- Confirm which of Ch.35's Blocking items must close before NCIE-005/006/007/009/011/012/014/015/016 may begin detailed implementation versus which may proceed against the Proposed Design Default provisionally.

Traceability: Closes the NCIE-001 → NCIE-002 → NCIE-003 → NCIE-004 authority chain (front matter §0.1).